

HTML Image

HTML img tag is used to display image on the web page. HTML img tag is an empty tag that contains attributes only, closing tags are not used in HTML image element.

Let's see an example of HTML image.

1. **<h2>HTML Image Example</h2>**
2. ****

Output:



Attributes of HTML img tag

The src and alt are important attributes of HTML img tag. All attributes of HTML image tag are given below.

1) src

It is a necessary attribute that describes the source or path of the image. It instructs the browser where to look for the image on the server.

The location of image may be on the same directory or another server.

2) alt

The alt attribute defines an alternate text for the image, if it can't be displayed. The value of the alt attribute describe the image in words. The alt attribute is considered good for SEO prospective.

3) width

It is an optional attribute which is used to specify the width to display the image. It is not recommended now. You should apply CSS in place of width attribute.

4) height

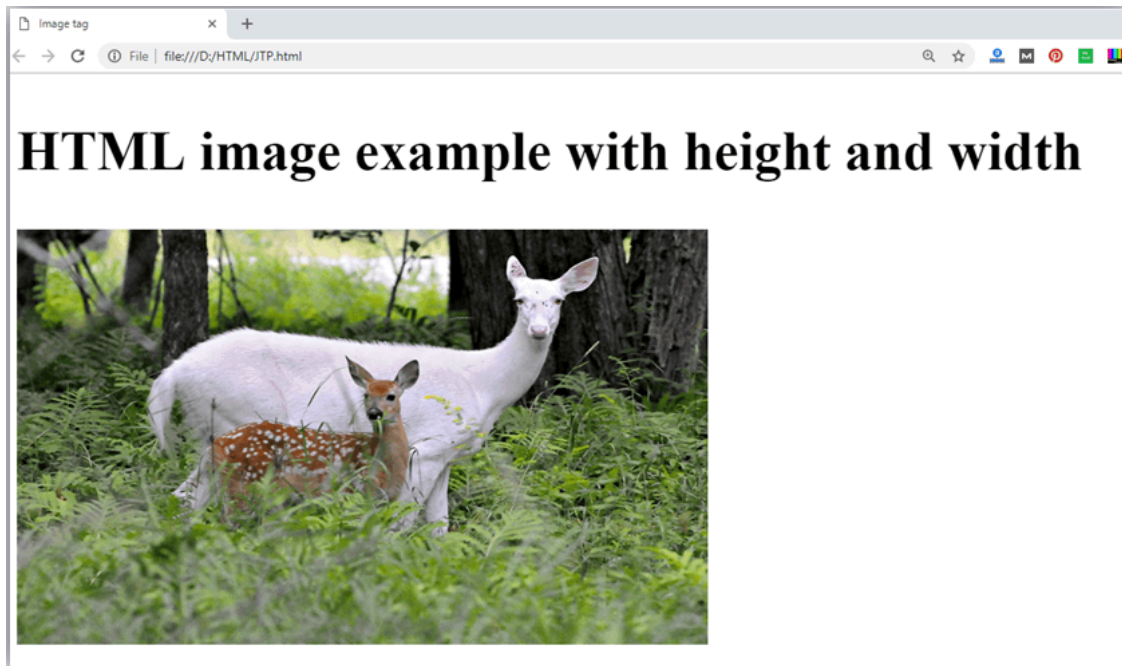
It h3 the height of the image. The HTML height attribute also supports iframe, image and object elements. It is not recommended now. You should apply CSS in place of height attribute.

Use of height and width attribute with img tag

You have learnt about how to insert an image in your web page, now if we want to give some height and width to display image according to our requirement, then we can set it with height and width attributes of image.

Example:

1. ``



Use of alt attribute

We can use alt attribute with `<img>` tag. It will display an alternative text in case if image cannot be displayed on browser. Following is the example for alt attribute:

1. ``

Use `<img>` tag as a link

We can also link an image with other page or we can use an image as a link. To do this, put `<img>` tag inside the `<a>` tag.

Example:

1. `<a href="https://www.javatpoint.com/what-is-robotics"></a>`

Output:



HTML Ordered Lists

The HTML `<ol>` tag defines an ordered list. An ordered list can be numerical or alphabetical.

Ordered HTML List

An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag.

The list items will be marked with numbers by default:

EXP-

```
<ol>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Ordered HTML List - The Type Attribute

The type attribute of the `<ol>` tag, defines the type of the list item marker:

Type	Description
type="1"	The list items will be numbered with numbers (default)
type="A"	The list items will be numbered with uppercase letters
type="a"	The list items will be numbered with lowercase letters
type="I"	The list items will be numbered with uppercase roman numbers
type="i"	The list items will be numbered with lowercase roman numbers

Numbers:

```
<ol type="1">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

Uppercase Letters:

```
<ol type="A">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

Lowercase Letters:

```
<ol type="a">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

Uppercase Roman Numbers:

```
<ol type="I">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

Lowercase Roman Numbers:

```
<ol type="i">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

Control List Counting

By default, an ordered list will start counting from 1. If you want to start counting from a specified number, you can use the start attribute:

Exp-

```
<ol start="50">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

Nested HTML Lists

Lists can be nested (list inside list):

EXP---

```
<ol>  
  <li>Coffee</li>  
  <li>Tea  
    <ol>  
      <li>Blacktea</li>  
      <li>Greentea</li>  
    </ol>  
  </li>  
</ol>
```

```
</li>  
<li>Milk</li>  
</ol>
```

How to create an unordered list with image bullets in HTML

An ordered list is numbered and an unordered list is not numbered it can be created with the tag `<ul></ul>` and define the items of the list using the tag `<li></li>`. We can create 4 types of unordered list in HTML –

- **disc** – This creates an unordered list marked to a bullet (default).
- **circle** – This creates an unordered list marked to a circle.
- **square** – This creates an unordered list marked to a square.
- **none** – This creates an unordered list without any marker.


```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
  <ul style="list-style-image: url('image_path')">
    <li>list item1</li>
    <li>list item2</li>
  </ul>
</body>
</html>
```

Syntax

Following is the syntax to create an unordered list with square bullets in HTML.

```
<ul style="list-style-image: url(image.png)">
  <li>Item in list...</li>
  <li>Item in list...</li>
  <li>Item in list...</li>
</ul>
```

Example 1

Given below is an example to create an unordered list with image bullets in HTML.

HTML.

```
<!DOCTYPE html>
<html>
```

```

<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
</head>
<body>
  <ul style="list-style-image: url(images/11.png)">
    <li>Abdul</li>
    <li>Jason</li>
    <li>Yadav</li>
  </ul>
</body>
</html>

```

Example 2

Let us look at another example to create an unordered list with image bullets –

```

<!DOCTYPE html>
<html>
<head>
  <title>HTML Unordered List</title>
</head>
<body>
  <h2>Tutorials</h2>
  <p>The list of tutorials:</p>
  <ul style="list-style-image:
url('/theme/css/icons/image-editor.png');">
    <li>Java</li>
    <li>C++</li>
    <li>Ruby</li>
  </ul>

```

```
</body>  
</html>
```

Horizontal List with CSS

HTML lists can be styled in many different ways with CSS.

One popular way is to style a list horizontally, to create a navigation menu:

Example

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
ul {  
  list-style-type: none;  
  margin: 0;  
  padding: 0;  
  overflow: hidden;  
  background-color: #333333;  
}  
  
li {  
  float: left;  
}  
  
li a {  
  display: block;  
  color: white;  
  text-align: center;  
  padding: 16px;  
  text-decoration: none;  
}
```

```
li a:hover {
  background-color: #111111;
}
</style>
</head>
<body>

<ul>
  <li><a href="#home">Home</a></li>
  <li><a href="#news">News</a></li>
  <li><a href="#contact">Contact</a></li>
  <li><a href="#about">About</a></li>
</ul>

</body>
</html>
```

OUTPUT-----Navigation Menu

In this example, we use CSS to style the list horizontally, to create a navigation menu:



Inserting Graphics in HTML

We can use different fonts and lists in an HTML document. Similarly, we can add graphics to make the document look more attractive. Web

browsers support a number of graphic formats. Some of the most widely used formats are:

- **Graphics Interchange Format(GIF):** GIF is the best format displaying images designed with a graphics program. This format uses a maximum of 256 colors and a combination of these to create more colors.
- **Joint Photographic Expert Group(JPEG):** JPEG is the best format for photographs as it contains 1 million colors.
- **Portable Network Graphics(PNG):** The format is best for images with transparency or the low color count.

Use of Tag

The tag specifies an image to be displayed in an HTML document. It is an element as it does not have an OFF tag.

The image tag has the following attributes in the table:

Attribute	Description	Values	Example
src	Specifies the URL image	URL	
alt	Specifies the alternative text for an image. This is displayed when the graphics feature is turned off in the browser or while The image is being downloaded. In some browsers, the ALT text is displayed as a tooltip for the image.	text	
height	Specifies the height of an image	value in pixels	

width	Specifies the width of an image	Value in pixels	
-------	---------------------------------	-----------------	---------------------------------

The following HTML code will help you understand the tag better. The resulting web page is shown in the output.

Inserting an image with CSS

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
  <style>
```

```
    image{border:4px solid orange; width: 120px;height: 100px;}
```

```
    h1{text-transform:uppercase}
```

```
  </style>
```

```
<head>
```

```
  <title>
```

```
    HTML frame scrolling Attribute
```

```
  </title>
```

```
</head>
```

```
<body>
```

```
<h1>INSERTING IMAGE</h1>
```

```

```

```
</body>
```

```
</html>
```

Output:



HTML Table

HTML table tag is used to display data in tabular form (row * column). There can be many columns in a row.

We can create a table to display data in tabular form, using <table> element, with the help of <tr> , <td>, and <th> elements.

In Each table, table row is defined by <tr> tag, table header is defined by <th>, and table data is defined by <td> tags.

HTML tables are used to manage the layout of the page e.g. header section, navigation bar, body content, footer section etc. But it is recommended to use div tag over table to manage the layout of the page .

HTML Table Tags

Tag	Description
<table>	It defines a table.
<tr>	It defines a row in a table.
<th>	It defines a header cell in a table.
<td>	It defines a cell in a table.
<caption>	It defines the table caption.
<colgroup>	It specifies a group of one or more columns in a table for formatting.
<col>	It is used with <colgroup> element to specify column properties for each column.
<tbody>	It is used to group the body content in a table.
<thead>	It is used to group the header content in a table.
<tfooter>	It is used to group the footer content in a table.

HTML Table Example

Let's see the example of HTML table tag. Its output is shown above.

1. **<table>**
2. **<tr><th>First_Name</th><th>Last_Name</th><th>Marks</th></tr>**
3. **<tr><td>Sonoo</td><td>Jaiswal</td><td>60</td></tr>**
4. **<tr><td>James</td><td>William</td><td>80</td></tr>**
5. **<tr><td>Swati</td><td>Sironi</td><td>82</td></tr>**
6. **<tr><td>Chetna</td><td>Singh</td><td>72</td></tr>**
7. **</table>**

OUTPUT



The screenshot shows an online code editor with a green header bar containing icons for home, download, print, and a menu, along with a 'Run' button and the text 'Online Code Editor'. The editor area is split into two panes. The left pane contains the following HTML code:

```
<!DOCTYPE>
<html>
<body>
<table>
<tr><th>First_Name</th><th>Last_Name</th><th>Marks</th></tr>
<tr><td>Sonoo</td><td>Jaiswal</td><td>60</td></tr>
<tr><td>James</td><td>William</td><td>80</td></tr>
<tr><td>Swati</td><td>Sironi</td><td>82</td></tr>
<tr><td>Chetna</td><td>Singh</td><td>72</td></tr>
</table>
</body>
</html>
```

The right pane displays the rendered output of this code, which is a table with three columns: 'First_Name', 'Last_Name', and 'Marks'. The table contains four data rows:

First_Name	Last_Name	Marks
Sonoo	Jaiswal	60
James	William	80
Swati	Sironi	82
Chetna	Singh	72

Frame Creation and Layouts

HTML frames allow authors to present documents in multiple views, which may be independent windows or subwindows. Multiple views offer designers a way to keep certain information visible, while other views are scrolled or replaced. For example, within the same window, one frame might display a static banner, a second a navigation menu, and a third the main document that can be scrolled through or replaced by navigating in the second frame.

HTML <frame> tag (Not supported in HTML5)

HTML <frame> tag define the particular area within an HTML file where another HTML web page can be displayed.

A <frame> tag is used with <frameset>, and it divides a webpage into multiple sections or frames, and each frame can contain different web pages.

Note: Do not use HTML <frame> tag as it is not supported in HTML5, instead you can use <iframe> or <div> with CSS to achieve similar effects in HTML.

Syntax

1. `< frame src = "URL" >`

Following are some specifications about the HTML <frame> tag

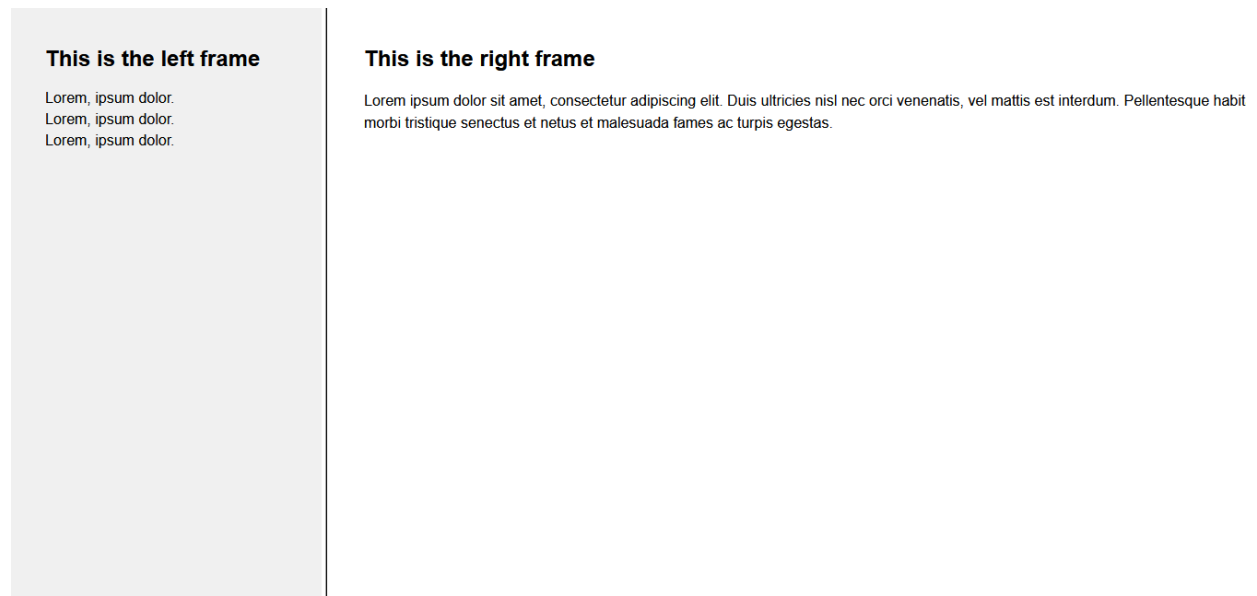
Display	Block
Start tag/End tag	Start tag(required), End tag(forbidden)
Usage	Frames

Example 1

Create Vertical frames:

1. `<!DOCTYPE html>`
2. `<html>`
3. `<head>`
4. `<title>Frame tag</title>`
5. `</head>`
6. `<frameset cols="25%,50%,25%">`
7. `<frame src="frame1.html" >`
8. `<frame src="frame2.html">`
9. `<frame src="frame3.html">`
10. `</frameset>`
11. `</html>`

Output



Frame1.html

Example 2:

Create Horizontal frames:

1. `<!DOCTYPE html>`
2. `<html>`
3. `<head>`
4. `<title>Frame tag</title>`
5. `</head>`
6. `<frameset rows="30%, 40%, 30%">`
7. `<frame name="top" src="frame1.html" >`
8. `<frame name="main" src="frame2.html">`
9. `<frame name="bottom" src="frame3.html">`
10. `</frameset>`
11. `</html>`

Output:

This is the Top frame

This is the buttom frame

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis ultricies nisl nec orci venenatis, vel mattis est interdum. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.

Attribute

Tag-specific attribute

Attribute	Value	Description
frameborder	0 1	It specifies whether to display a border around the frame or not, and its default value is 1
longdesc	URL	It specifies a page which contains the long description of the content of the frame.
marginheight	pixels	It specifies the top and bottom margins of the frame.
marginwidth	pixels	It defines the height of the margin between frames.
name	text	It is used to assign the name to the frame.
noresize	noresize	It is used to prevent resizing of the frame by the user.

scrolling	yes no auto	It specifies the existence of the scrollbar for overflowing content.
src	URL	It specifies the URL of the document which we want to display in a frame.

Working with Forms and Menus

What is an HTML Form?

An HTML or a Web form helps collect user input. HTML forms can have different fields, such as text areas, text boxes, radio buttons, checkboxes, drop-down lists, file uploads, etc.

The collected data can be validated on the client browser using JavaScript. After validation of form data on the client-side, the user clicks on the Submit button in the form. After this, data is sent to the server for further processing.

HTML <input> Element

We use the HTML <input> element to create form fields and receive input from the user. We can use various input fields to take different information from the user. Using different **Type attributes**, we can display an **<input> element** in various ways.

Here is an example to take text input from the user:

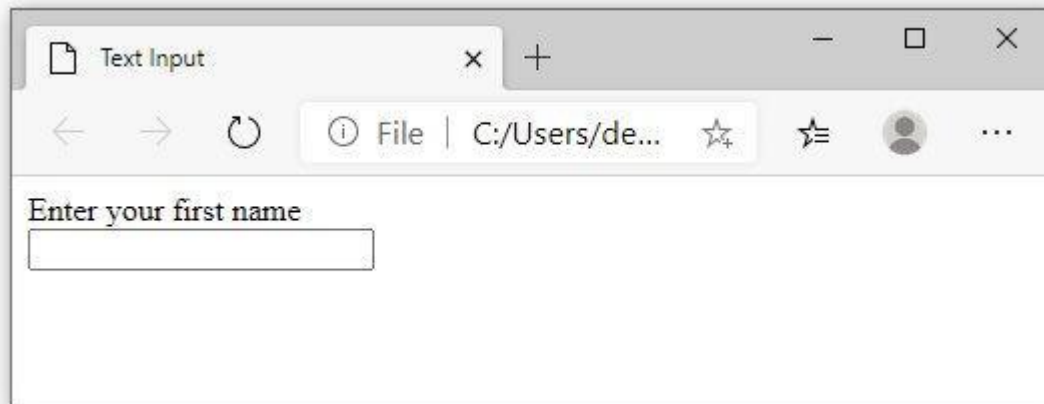
Output:

Output:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Text Input</title>
  </head>
  <body>
    <form>
      Enter your first name <br>
      <input type="text" name="username">
    </form>
```

```
</body>
</html>
```

Copy code



Common HTML Form Tags

The following are some of the commonly used HTML tags to create web forms:

Tag	Description
<form>	This tag defines an HTML form to receive input from the user.
<input>	Defines an input control.
<textarea>	Defines a multi-line input control.
<label>	Creates a caption for input elements.
<select> & <option>	Create a drop-down menu with various options. The <select> tag defines the drop-down, while the <option> tag represents all the options in the list.
<optgroup>	Creates a drop-down list of related options.
<fieldset>	Groups the related element in a web form.
<legend>	It defines a caption for the <fieldset> element.
<button>	Creates a clickable button.

<output>	It is used for defining the result of a calculation.
----------	--

HTML Form Controls

An HTML form has form controls, such as text fields, text areas, checkboxes, buttons, etc. It is a user interface control that acts as a point of connection between the user and the server. Here are some of the commonly used HTML form controls to gather data:

- Text Input Controls
- Password in an HTML Form
- Radio Button Control
- Checkbox Control
- Submit Button Control
-

Let us understand each of these Form controls using the different Type attributes in HTML <form> tag.

1. Text Input Controls

The <input type="text"> control defines a single-line text input. We can use the HTML <textarea> tag for creating multi-line input control, if the user input may be longer than a single line.

Let us look at an example to take the user's first name and last name as input.

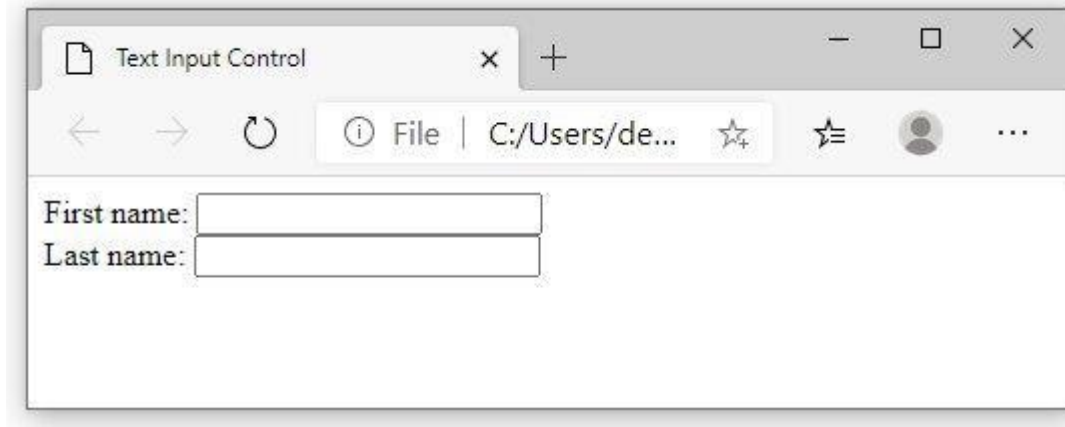
Example of a form with Text input fields:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Text Input Control</title>
  </head>
  <body>
    <form >
      First name: <input type = "text" name = "fname" /><br>
      Last name: <input type = "text" name = "lname" />
    </form>
  </body>
```

```
</html>
```

Copy code

Output:

A screenshot of a web browser window. The title bar says 'Text Input Control'. The address bar shows 'File | C:/Users/de...'. The main content area displays a simple form with two text input fields. The first field is preceded by the label 'First name:' and the second field is preceded by 'Last name:'. Both fields are empty.

2. Password in an HTML Form

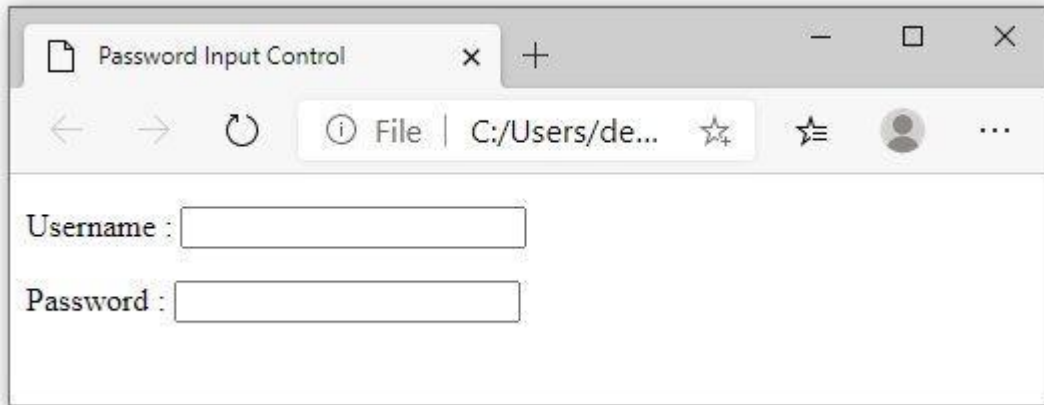
We use the type value as 'password' to take the password as input. The password entered by the user will not be visible in the password field control.

Example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Password Input Control</title>
  </head>
  <body>
    <form>
      <p>
        <label>Username : <input type="text" /></label>
      </p>
      <p>
        <label>Password : <input type="password" /></label>
      </p>
    </form>
  </body>
</html>
```

Copy code

Output:



3. Radio Button Control

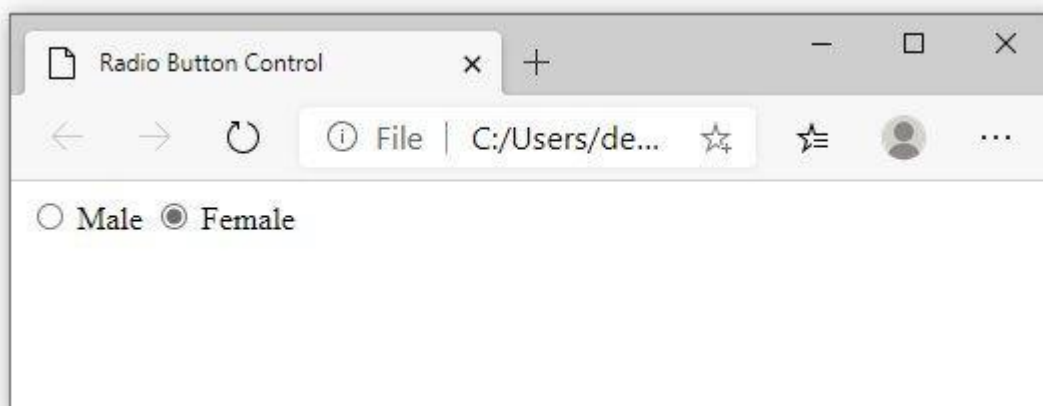
Radio buttons let users select one option out of many options. We can create a Radio Button control using the HTML `<input>` tag and the type attribute will be radio. `<input type="radio">` defines a radio button.

Example of an HTML form with radio buttons:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Radio Button Control</title>
  </head>
  <body>
    <form>
      <input type = "radio" name = "Gender" value = "Male"> Male
      <input type = "radio" name = "Gender" value = "Female"> Female
    </form>
  </body>
</html>
```

Copy code

Output:



4. Checkbox Control

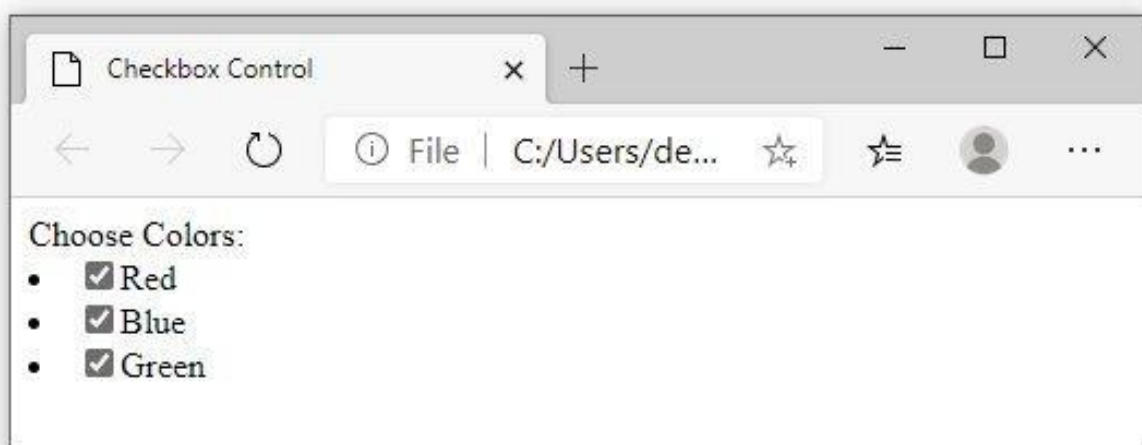
The checkbox is used when more than one option is to be selected from the given options. We can create it using the HTML `<input>` tag and set the type attribute to the checkbox.

Example:

Below is an example to create a form with checkboxes.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Checkbox Control</title>
  </head>
  <body>
    <form>
      Choose Colors:<br>
      <li><input type="checkbox" name="color" value="red" />Red</li>
      <li><input type="checkbox" name="color" value="blue" />Blue</li>
      <li><input type="checkbox" name="color" value="green" />Green</li>
    </form>
  </body>
</html>
```

Output:



HTML Forms Example

Let us look at an example to create a basic employee detail form that will take input data, including First Name, Last Name, Gender, Employee ID, Designation, and Phone Number.

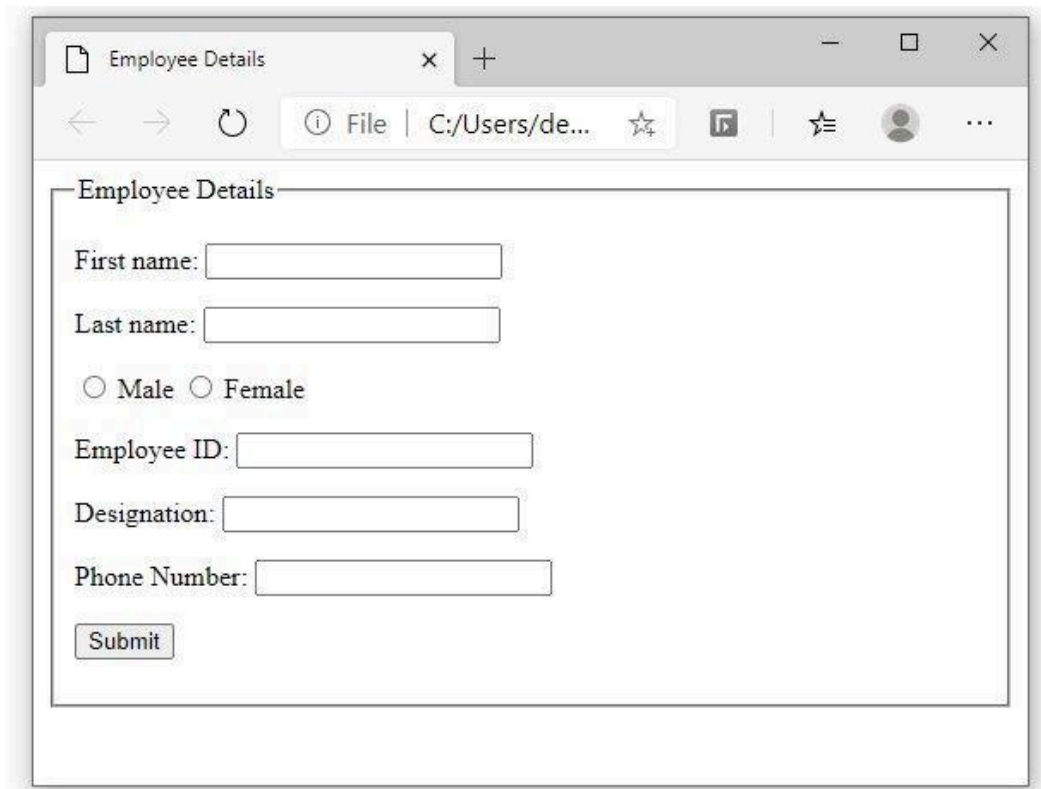
Create an HTML Form to Input the Basic Details of an Employee

To create an Employee detail form, we will use a combination of tags, such as `<legend>` to specify the caption; `<input>` tag for First Name, Last Name, and Phone Number; radio button for gender; `<button>` to submit, etc.

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Employee Details</title>
</head>
<body>
<form>
<fieldset>
<legend>Employee Details</legend>
<p>
First name: <input type = "text" name = "fname" />
</p>
<p>
Last name: <input type = "text" name = "lname" />
</p>
<p>
<input type = "radio" name = "Gender" value = "Male"> Male
<input type = "radio" name = "Gender" value = "Female"> Female
</p>
<p>
Employee ID: <input type = "text" name = "ID" />
</p>
<p>
Designation: <input type = "text" name = "ID" />
</p>
<p>
```

```
Phone Number: <input type = "text" name = "phone" />
</p>
<p>
  <input type = "submit" name = "submit" value = "Submit" />
</p>
</fieldset>
</form>
</body>
</html>
```

Output:



The screenshot shows a web browser window with a single tab titled 'Employee Details'. The address bar shows the file path 'C:/Users/de...'. The form is titled 'Employee Details' and contains the following fields and controls:

- First name:
- Last name:
- Gender: ☐ Male ☐ Female
- Employee ID:
- Designation:
- Phone Number:
- Submit:

DHTML

DHTML stands for Dynamic HTML, it is totally different from HTML. The browsers which support the dynamic HTML are some of the versions of Netscape Navigator and Internet Explorer of version higher than 4.0. The DHTML is based on the properties of the HTML, javascript, CSS, and DOM (Document Object Model which is used to access individual elements of a document) which helps in making dynamic content. It is the combination of HTML, CSS, JS, and DOM. The DHTML make use of Dynamic object model to make changes in settings and also in properties and methods. It also makes uses of Scripting and it is also part of earlier computing trends.

DHTML allows different scripting languages in a web page to change their variables, which enhance the effects, looks and many others functions after the whole page have been fully loaded or under a view process, or otherwise static HTML pages on the same. But in true ways, there is noting that as dynamic in DHTML, there is only the enclosing of different technologies like CSS, HTML, JS, DOM, and different sets of static languages which make it as dynamic.

DHTML is used to create interactive and animated web pages that are generated in real-time, also known as dynamic web pages so that when such a page is accessed, the code within the page is analyzed on the web server and the resulting HTML is sent to the client's web browser.

HTML: HTML stands for Hypertext Markup Language and it is a client-side markup language. It is used to build the block of web pages.

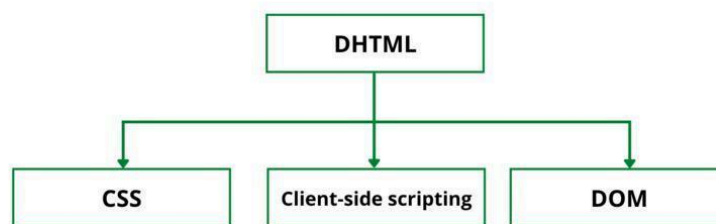
Javascript: It is a Client-side Scripting language. Javascript is supported by most of the browser, also have cookies collection to determine the user needs.

CSS: The abbreviation of CSS is Cascading Style Sheet. It helps in the styling of the web pages and helps in designing of the pages. The CSS rules for DHTML will be modified at different levels using JS with event handlers which adds a significant amount of dynamism with very little

code.

DOM: It is known as a Document Object Model which act as the weakest links in it. The only defect in it is that most of the browser does not support DOM. It is a way to manipulate the static contents.

DHTML is not a technology; rather, it is the combination of three different technologies, client-side scripting (JavaScript or VBScript), cascading style sheets and document object model.



Note: Many times DHTML is confused with being a language like HTML but it is not. It must be kept in mind that it is an interface or browsers enhancement feature which makes it possible to access the object model through Javascript language and hence make the webpage more interactive.

Key Features: Following are the some major key features of DHTML:

- Tags and their properties can be changed using DHTML.
- It is used for real-time positioning.

- Dynamic fonts can be generated using DHTML.
- It is also used for data binding.
- It makes a webpage dynamic and be used to create animations, games, applications along with providing new ways of navigating through websites.
- The functionality of a webpage is enhanced due to the usage of low-bandwidth effect by DHTML.
-
- DHTML also facilitates the use of methods, events, properties, and codes.

Uses DHTML

Following are the uses of DHTML (Dynamic HTML):

- It is used for designing the animated and interactive web pages that are developed in real-time.
- DHTML helps users by animating the text and images in their documents.
- It allows the authors for adding the effects on their pages.
- It also allows the page authors for including the drop-down menus or rollover buttons.
- This term is also used to create various browser-based action games.
- It is also used to add the ticker on various websites, which needs to refresh their content automatically.

Features of DHTML

Following are the various characteristics or features of DHTML (Dynamic HTML):

- Its simplest and main feature is that we can create the web page dynamically.
- **Dynamic Style** is a feature, that allows the users to alter the font, size, color, and content of a web page.
- It provides the facility for using the events, methods, and properties. And, also provides the feature of code reusability.

- o It also provides the feature in browsers for data binding.
- o Using DHTML, users can easily create dynamic fonts for their web sites or web pages.
- o With the help of DHTML, users can easily change the tags and their properties.
- o The web page functionality is enhanced because the DHTML uses low-bandwidth effect.

Advantages:

- Size of the files are compact in compared to other interactional media like Flash or Shockwave, and it downloads faster.
- It is supported by big browser manufacturers like Microsoft and Netscape.
- Highly flexible and easy to make changes.
- Viewer requires no extra plug-ins for browsing through the webpage that uses DHTML, they do not need any extra requirements or special software to view it.
- User time is saved by sending less number of requests to the server. As it is possible to modify and replace elements even after a page is loaded, it is not required to create separate pages for changing styles which in turn saves time in building pages and also reduces the number of requests that are sent to the server.
- It has more advanced functionality than a static HTML. it is capable of holding more content on the web page at the same time.

Disadvantages:

- It is not supported by all the browsers. It is supported only by recent browsers such as Netscape 6, IE 5.5, and Opera 5 like browsers.
- Learning of DHTML requires a lot of pre-requisites languages such as HTML, CSS, JS, etc should be known to the designer before starting with DHTML which is a long and time-consuming in itself.
- Implementation of different browsers are different. So if it worked in one browser, it might not necessarily work the same way in another browser.

- Even after being great with functionality, DHTML requires a few tools and utilities that are some expensive. For example, the DHTML text editor, Dreamweaver. Along with it the improvement cost of transferring from HTML to DHTML makes cost rise much higher.
 - Its simplest and main feature is that we can create the web page dynamically.
 - **Dynamic Style** is a feature, that allows the users to alter the font, size, color, and content of a web page.
 - It provides the facility for using the events, methods, and properties. And, also provides the feature of code reusability.
 - It also provides the feature in browsers for data binding.
 - Using DHTML, users can easily create dynamic fonts for their web sites or web pages.
 - With the help of DHTML, users can easily change the tags and their properties.
 - The web page functionality is enhanced because the DHTML uses low-bandwidth effect.
 -

Difference between HTML and DHTML

Following table describes the differences between HTML and DHTML:

HTML (Hypertext Markup language)	DHTML (Dynamic Hypertext Markup language)
1. HTML is simply a markup language.	1. DHTML is not a language, but it is a set of technologies of web development.
2. It is used for developing and creating web pages.	2. It is used for creating and designing the animated and interactive web sites or pages.
3. This markup language creates static web pages.	3. This concept creates dynamic web pages.

4. It does not contain any server-side scripting code.	4. It may contain the code of server-side scripting.
5. The files of HTML are stored with the .html or .htm extension in a system.	5. The files of DHTML are stored with the .dhtm extension in a system.
6. A simple page which is created by a user without using the scripts or styles called as an HTML page.	6. A page which is created by a user using the HTML, CSS, DOM, and JavaScript technologies called a DHTML page.
7. This markup language does not need database connectivity.	7. This concept needs database connectivity because it interacts with users.

DHTML JavaScript

JavaScript can be included in HTML pages, which creates the content of the page as dynamic. We can easily type the JavaScript code within the <head> or <body> tag of a HTML page. If we want to add the external source file of JavaScript, we can easily add using the <src> attribute.

Following are the various examples, which describes how to use the JavaScript technology with the DHTML:

Document.write() Method

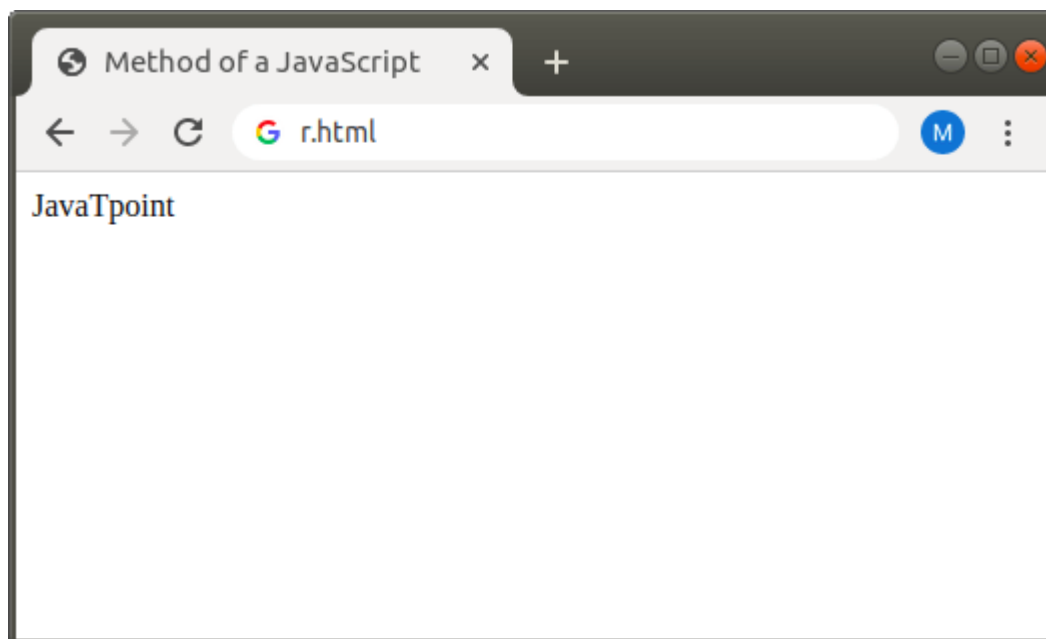
The document.write() method of JavaScript, writes the output to a web page.

Example 1: The following example simply uses the **document.write()** method of JavaScript in the DHTML. In this example, we type the JavaScript code in the <body> tag.

1. <HTML>
2. <head>
3. <title>
4. Method of a JavaScript

5. `</title>`
6. `</head>`
7. `<body>`
8. `<script type="text/javascript">`
9. `document.write("JavaTpoint");`
10. `</script>`
11. `</body>`
12. `</html>`

Output:



JavaScript and HTML event

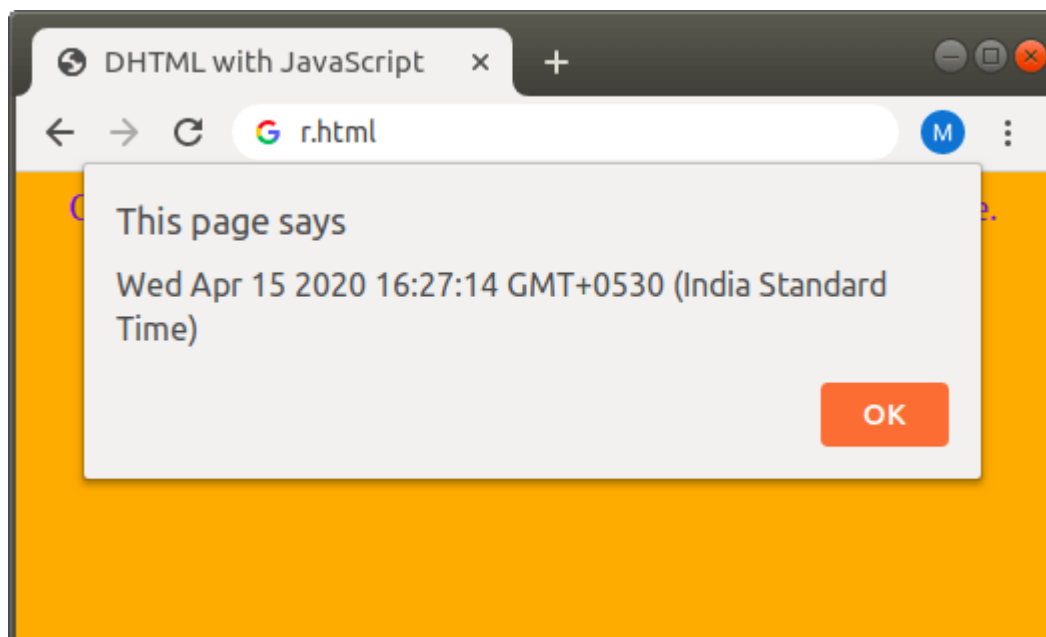
A JavaScript code can also be executed when some event occurs. Suppose, a user clicks an HTML element on a webpage, and after clicking, the JavaScript function associated with that HTML element is automatically invoked. And, then the statements in the function are performed.

Example 1: The following example shows the current date and time with the JavaScript and HTML event (Onclick). In this example, we type the JavaScript code in the `<head>` tag.

1. `<html>`
2. `<head>`

```
3. <title>
4. DHTML with JavaScript
5. </title>
6. <script type="text/javascript">
7. function dateandtime()
8. {
9. alert(Date());
10.}
11.</script>
12.</head>
13.<body bgcolor="orange">
14.<font size="4" color="blue">
15.<center> <p>
16.Click here # <a href="#" onClick="dateandtime();"> Date and Time </a>
17.# to check the today's date and time.
18.</p> </center>
19.</font>
20.</body>
21.</html>
```

Output:



JavaScript and HTML DOM

With version 4 of HTML, JavaScript code can also change the inner content and attributes of the HTML event.

Example 1: This example checks the Grade of a student according to the percentage criteria with the JavaScript and HTML DOM. In this example, we type the code of a JavaScript in the <body> tag.

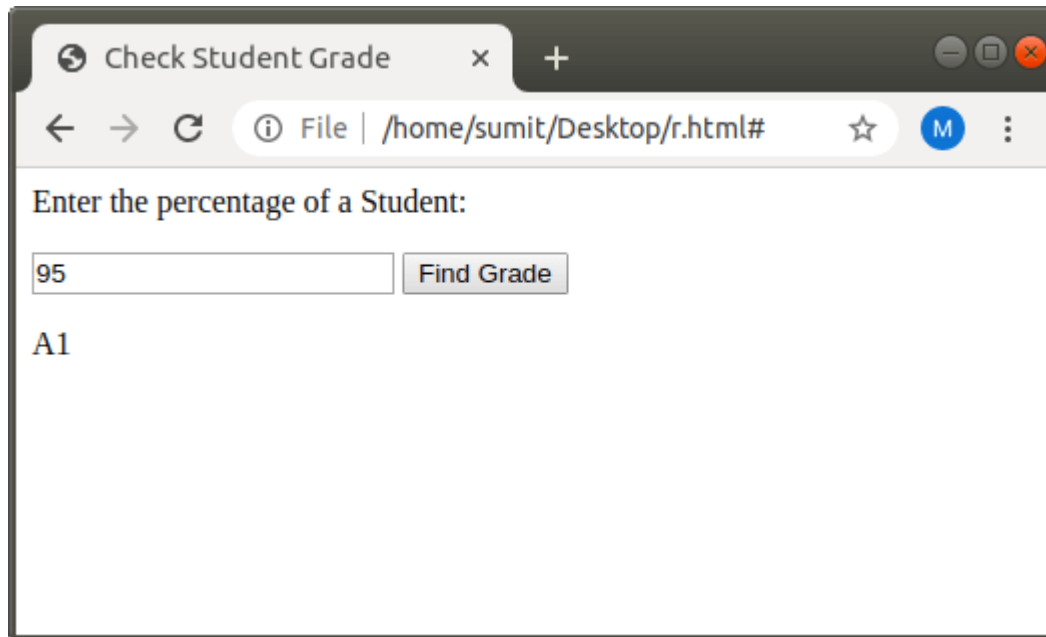
```
1.  <html>
2.    <head>
3.      <title> Check Student Grade
4.    </title>
5.  </head>
6.
7.  <body>
8.    <p>Enter the percentage of a Student:</p>
9.    <input type="text" id="percentage">
10.  <button type="button" onclick="checkGrade()">
11.    Find Grade
12.  </button>
13. <p id="demo"></p>
14. <script type="text/javascript">
15.   function checkGrade() {
16.     var x,p, text;
17.     p = document.getElementById("percentage").value;
18.
19.     x=parseInt(p);
20.
21.
22.     if (x>90 && x <= 100) {
23.       document.getElementById("demo").innerHTML = "A1";
24.     } else if (x>80 && x <= 90) {
25.       document.getElementById("demo").innerHTML = "A2";
26.     } else if (x>70 && x <= 80) {
27.       document.getElementById("demo").innerHTML = "A3";
```

```

28.     }
29.     }
30.     </script>
31. </body>
32.</html>

```

Output:



Example 2: The following example uses the DHTML CSS for changing the style of the HTML element:

```

1. <html>
2. <head>
3.   <title>
4.     changes the particular HTML element example
5.   </title>
6. </head>
7. <body>
8.   <p id="demo"> This text changes color when click on the following different buttons. </p>
9.   <button onclick="change_Color('green');"> Green </button>
10.  <button onclick="change_Color('blue');"> Blue </button>
11.  <script type="text/javascript">
12.

```

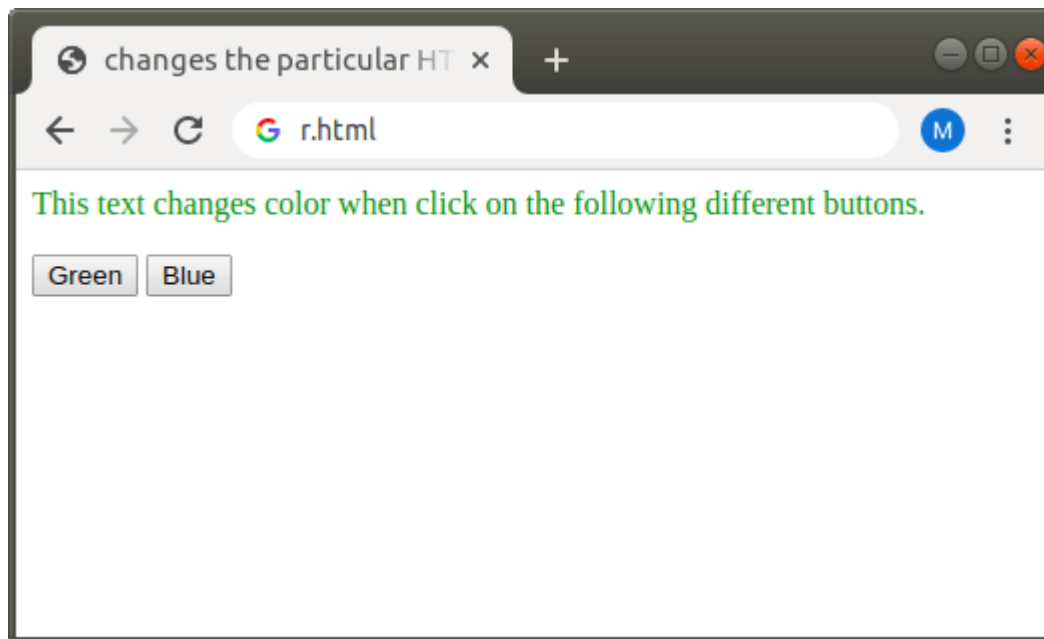
```
13. function change_Color(newColor) {  
14.   var element = document.getElementById('demo').style.color = newColor;  
15. }  
16. </script>  
17. </body>  
18. </html>
```

Test it Now

Output:

ADVERTISEMENT

ADVERTISEMENT



Explanation:

In the above code, we used the **var element = document.getElementById('demo').style.color = newColor;** statement, which changes the color of a text as green and blue according to the buttons.

DHTML Events

An event is defined as changing the occurrence of an object.

It is compulsory to add the events in the DHTML page. Without events, there will be no dynamic content on the HTML page. The event is a term in the HTML, which triggers the actions in the web browsers.

Suppose, any user clicks an HTML element, then the JavaScript code associated with that element is executed. Actually, the event handlers catch the events performed by the user and then execute the code.

Example of events:

1. Click a button.
2. Submitting a form.
3. An image loading or a web page loading, etc.

Following table describes the Event Handlers used in the DHTML:

S.No	Event	When it occurs
1.	onabort	It occurs when the user aborts the page or media file loading.
2.	onblur	It occurs when the user leaves an HTML object.
3.	onchange	It occurs when the user changes or updates the value of an object.
4.	onclick	It occurs or triggers when any user clicks on an HTML element.
5.	ondblclick	It occurs when the user clicks on an HTML element two times together.
6.	onfocus	It occurs when the user focuses on an HTML element. This event handler works opposite to onblur.
7.	onkeydown	It triggers when a user is pressing a key on a keyboard device. This event handler works for all the keys.
8.	onkeypress	It triggers when the users press a key on a keyboard. This event handler is not triggered for all the keys.
9.	onkeyup	It occurs when a user released a key from a keyboard after pressing on an object or element.
10.	onload	It occurs when an object is completely loaded.
11.	onmousedown	It occurs when a user presses the button of a mouse over an HTML element.

12.	onmousemove	It occurs when a user moves the cursor on an HTML object.
13.	onmouseover	It occurs when a user moves the cursor over an HTML object.
14.	onmouseout	It occurs or triggers when the mouse pointer is moved out of an HTML element.
15.	onmouseup	It occurs or triggers when the mouse button is released over an HTML element.
16.	onreset	It is used by the user to reset the form.
17.	onselect	It occurs after selecting the content or text on a web page.
18.	onsubmit	It is triggered when the user clicks a button after the submission of a form.
19.	onunload	It is triggered when the user closes a web page.

Following are the different examples using the different event handlers, which helps us to understand the concept of DHTML events:

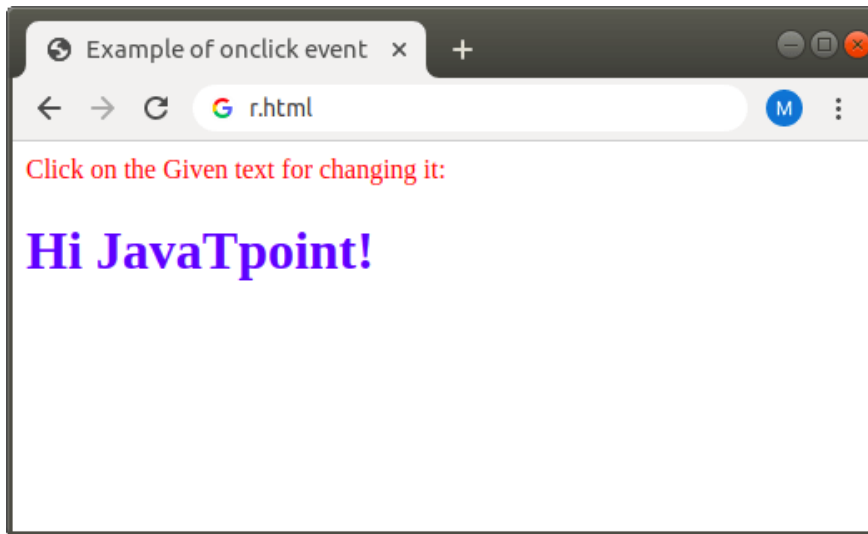
Example 1: This example uses the onclick event handler, which is used to change the text after clicking.

```

1. <html>
2. <head>
3. <title>
4. Example ofC onclick event
5. </title>
6. <script type="text/javascript">
7. function ChangeText(ctext)
8. {
9.   ctext.innerHTML=" Hi JavaTpoint! ";
10. }
11. </script>
12. </head>
13. <body>
14. <font color="red"> Click on the Given text for it: <br>
15. </font>
16. <font color="blue">
17. <h1 onclick="ChangeText(this)"> Hello World! </h1>
18. </font>
19. </body>
20. </html>
    changing

```

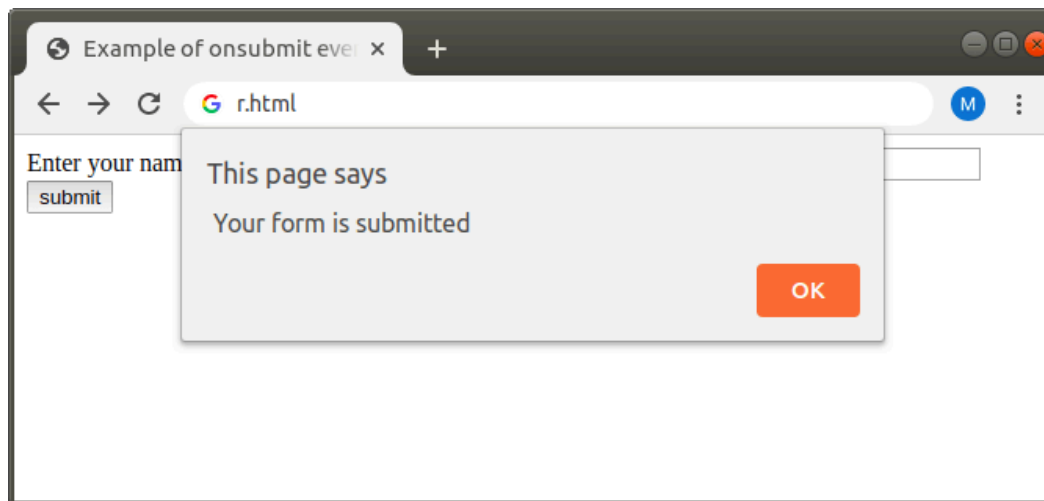
Output:



Example 2: This example uses the onsubmit event handler, which gives an alert after clicking on a submit button.z

1. `<html>`
2. `<head>`
3. `<title>`
4. Example of onsubmit event
5. `</title>`
6. `</head>`
7. `<body>`
8. `<form onsubmit="Submit_Form()">`
9. `<label> Enter your name: </label>`
10. `<input type="text">`
11. `<label> Enter your Roll no: </label>`
12. `<input type="Number">`
13. `<input type="submit" value="submit">`
14. `</form>`
15. `<script type="text/javascript">`
16. `function Submit_Form()`
17. `{`
18. `alert(" Your form is submitted");`
19. `}`
20. `</script>`
21. `</body>`
22. `</html>`

Output:



DHTML DOM

DHTML DOM stands for Dynamic HTML Document Object Model.

It is a w3c standard, which is a standard interface of programming for HTML. It is mainly used for defining the objects and properties of all elements in HTML. It also defines the methods for accessing the HTML elements.

What is the HTML DOM?

DOM is a programming interface for web documents. It allows us to modify HTML or XML document using functions. In this guide, we specifically cover HTML DOM, that allows us to manipulate HTML tags, attributes or elements using JavaScript.

We have covered DOM, its properties, structure, methods with examples. Understanding DOM and its benefits allow developers to get complete idea of web development. Modify HTML and CSS elements with DOM JavaScript.

The Document Object Model (DOM) is a **programming interface** for **HTML(HyperText Markup Language)** and **XML(Extensible Markup Language)** documents. It defines the **logical structure** of documents and the way a document is accessed and manipulated.

*So basically Document Object Model is an **API** that represents and interacts with HTML or XML documents.*

The DOM is a **W3C** (World Wide Web Consortium) **standard** and it defines a standard for accessing documents.

The W3C Dom standard is divided into three different parts:

- **Core DOM** – standard model for all document types
- **XML DOM** – standard model for XML documents
- **HTML DOM** – standard model for HTML documents

HTML DOM

HTML DOM is a standard **object model** and programming interface for **HTML documents**. **HTML DOM** is a way to represent the webpage in a **structured hierarchical way** so that it will become easier for programmers and users to glide through the document.

With HTML DOM, we can easily access and manipulate tags, IDs, classes, attributes, or elements of HTML using commands or methods provided by the document object.

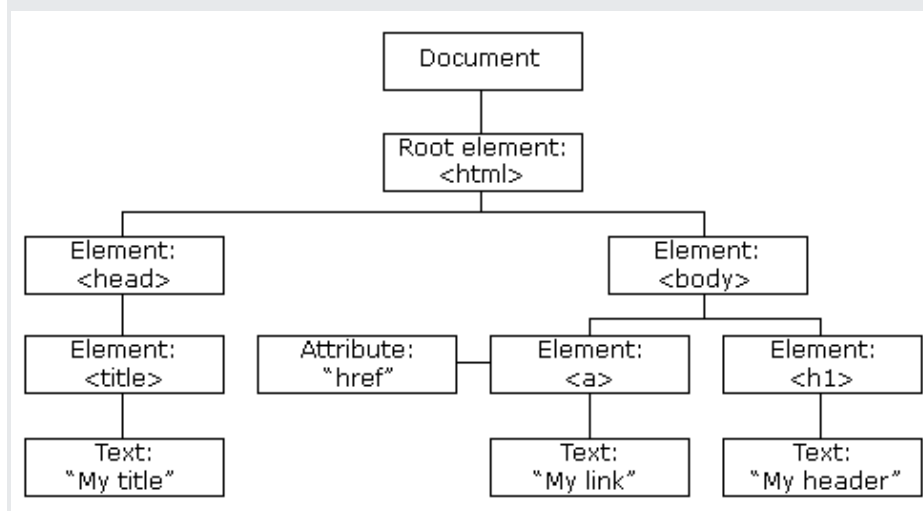
Using DOM JavaScript we get access to HTML as well as CSS of the web page and can also modify the behavior of the HTML elements.

The HTML DOM (Document Object Model)

When a web page is loaded, the browser creates a **Document Object Model** of the page.

The **HTML DOM** model is constructed as a tree of **Objects**:

The HTML DOM Tree of Objects



Finding HTML Elements

When you want to access HTML elements with JavaScript, you have to find the elements first.

There are a couple of ways to do this:

- Finding HTML elements by id

- Finding HTML elements by tag name
- Finding HTML elements by class name
- Finding HTML elements by CSS selectors
- Finding HTML elements by HTML object collections

Finding HTML Element by Id

The easiest way to find an HTML element in the DOM, is by using the element id.

This example finds the element with id="intro":

Example

```
var myElement = document.getElementById("intro");
```

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript HTML DOM</h2>
<p id="intro">Finding HTML Elements by Id</p>
<p>This example demonstrates the <b>getElementsById</b> method.</p>
<p id="demo"></p>
<script>
const element = document.getElementById("intro");
document.getElementById("demo").innerHTML =
"The text from the intro paragraph is: " + element.innerHTML;
</script>
</body>
</html>
```

OUTPUT

JavaScript HTML DOM

Finding HTML Elements by Id

This example demonstrates the **getElementsById** method.

The text from the intro paragraph is: Finding HTML Elements by Id

EXP-2 Finding HTML Elements by Tag Name

This example finds all <p> elements:

Example

```
var x = document.getElementsByTagName("p");
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript HTML DOM</h2>
```

```
<p>Finding HTML Elements by Tag Name.</p>
```

```
<p>This example demonstrates the <b>getElementsByTagName</b>  
method.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const element = document.getElementsByTagName("p");
```

```
document.getElementById("demo").innerHTML = 'The text in first  
paragraph (index 0) is: ' + element[0].innerHTML;
```

```
</script>
```

```
</body>
```

```
</html>
```

OUTPUT

JavaScript HTML DOM

Finding HTML Elements by Tag Name.

This example demonstrates the **getElementsByTagName** method.

The text in first paragraph (index 0) is: Finding HTML Elements by Tag Name.

EXP-3--- This example finds the element with id="main", and then finds all <p> elements inside "main":

Example

```
var x = document.getElementById("main");  
var y = x.getElementsByTagName("p");
```

```
<!DOCTYPE html>  
<html>  
<body>  
<h2>JavaScript HTML DOM</h2>  
<div id="main">  
<p>Finding HTML Elements by Tag Name</p>  
<p>This example demonstrates the <b>getElementsByTagName</b>  
method.</p>  
</div>  
<p id="demo"></p>  
<script>  
const x = document.getElementById("main");  
const y = x.getElementsByTagName("p");  
document.getElementById("demo").innerHTML =  
'The first paragraph (index 0) inside "main" is: ' + y[0].innerHTML;  
</script>  
</body>
```

</html>

OUTPUT

JavaScript HTML DOM

Finding HTML Elements by Tag Name

This example demonstrates the **getElementsByTagName** method.

The first paragraph (index 0) inside "main" is: Finding HTML Elements by Tag Name

EXP-4 Finding HTML Elements by Class Name

If you want to find all HTML elements with the same class name, use `getElementsByClassName()`.

This example returns a list of all elements with `class="intro"`.

Example

```
var x = document.getElementsByClassName("intro");
```

<!DOCTYPE html>

<html>

<body>

<h2>JavaScript HTML DOM</h2>

<p>Finding HTML Elements by Class Name.</p>

<p class="intro">Hello World!</p>

<p class="intro">This example demonstrates the
getElementsByClassName method.</p>

<p id="demo"></p>

<script>

const x = document.getElementsByClassName("intro");


```
document.getElementById("demo").innerHTML =  
'The first paragraph (index 0) with class="intro" is: ' + x[0].innerHTML;  
</script>  
</body>  
</html>
```

OUTPUT

JavaScript HTML DOM

Finding HTML Elements by Class Name.

Hello World!

This example demonstrates the **getElementsByClassName** method.

The first paragraph (index 0) with class="intro" is: Hello World!

EXP-5 Finding HTML Elements by CSS Selectors

If you want to find all HTML elements that matches a specified CSS selector (id, class names, types, attributes, values of attributes, etc), use the `querySelectorAll()` method.

This example returns a list of all `<p>` elements with class="intro".

Example

```
var x = document.querySelectorAll("p.intro");
```

```
<!DOCTYPE html>  
<html>  
<body>  
<h2>JavaScript HTML DOM</h2>  
<p>Finding HTML Elements by Query Selector</p>
```

<p class="intro">Hello World!.</p>

<p class="intro">This example demonstrates the querySelectorAll method.</p>

<p id="demo"></p>

<script>

const x = document.querySelectorAll("p.intro");

document.getElementById("demo").innerHTML =

'The first paragraph (index 0) with class="intro" is: ' + x[0].innerHTML;

</script>

</body>

</html>

OUTPUT

JavaScript HTML DOM

Finding HTML Elements by Query Selector

Hello World!.

This example demonstrates the **querySelectorAll** method.

The first paragraph (index 0) with class="intro" is: Hello World!.

EXP-6 Finding HTML Elements by HTML Object Collections

HTML object collections are also accessible:

- document.anchors
- document.forms
- document.images
- document.links
- document.scripts

<!DOCTYPE html>

```
<html>
<body>
<h2>Finding HTML Elements Using document.anchors</h2>
<a name="html">HTML Tutorial</a><br>
<a name="css">CSS Tutorial</a><br>
<a name="xml">XML Tutorial</a><br>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML =
"Number of anchors are: " + document.anchors.length;
</script>
</body>
</html>
```

OUTPUT

Finding HTML Elements Using document.anchors

HTML Tutorial

CSS Tutorial

XML Tutorial

Number of anchors are: 3

7- <!DOCTYPE html>

```
<html>
<body>
<h2>Finding HTML Elements Using document.forms</h2>
<form action="">
First name: <input type="text" name="fname" value="Donald">
<input type="submit" value="Submit">
```

```
</form>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML =
"Number of forms: " + document.forms.length;
</script>
</body>
</html>
```

Finding HTML Elements Using document.forms

Top of Form

First name:

Bottom of Form

Number of forms: 1

```
8- <!DOCTYPE html>
<html>
<body>
<h2>Finding HTML Elements Using document.scripts</h2>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML =
"Number of scripts: " + document.scripts.length;
</script>
</body>
</html>
```

Output--Finding HTML Elements Using document.scripts

Number of scripts: 1

This example illustrates the dom-manipulation

using *getElementById()* Method.

- HTML

```
<!DOCTYPE html>

<html>

<head>

    <title>DOM manipulation</title>

</head>

<body>

    <label>Enter Value 1: </label>

    <input type="text" id="val1" />

    <br />

    <br />

    <label>Enter Value 2: </label>

    <input type=".text" id="val2" />

    <br />
```

```
<button onclick="getAdd()">Click To Add</button>

<p id="result"></p>

<script type="text/javascript">

    function getAdd() {

        // Fetch the value of input with id val1

        const num1 = Number(document.getElementById("val1").value);

        // Fetch the value of input with id val2

        const num2 = Number(document.getElementById("val2").value);

        const add = num1 + num2;

        console.log(add);

        // Displays the result in paragraph using dom

        document.getElementById("result").innerHTML = "Addition : " +
add;

        // Changes the color of paragraph tag with red

        document.getElementById("result").style.color = "red";

    }

</script>

</body>
```

```
</html>
```

Output:

Enter Value 1

Enter Value 2

Addition : 30

Manipulating the Document objects using getElementById() Method

Why is DOM Required?

HTML is used to **structure** the web pages and **JavaScript** is used to add **behavior** to our web pages. When an HTML file is loaded into the browser, the JavaScript can not understand the HTML document directly. So it interprets and interacts with the Document Object Model (DOM), which is created by the browser based on the HTML document.

DOM is basically the representation of the same HTML document but in a tree-like structure composed of objects. JavaScript can not understand the tags(<h1>H</h1>) in HTML document but can understand object h1 in DOM.

JavaScript interprets DOM easily, using it as a bridge to access and manipulate the elements. DOM Javascript allow access to each of the objects (h1, p, etc) by using different functions.

The Document Object Model (DOM) is essential in web development for several reasons:

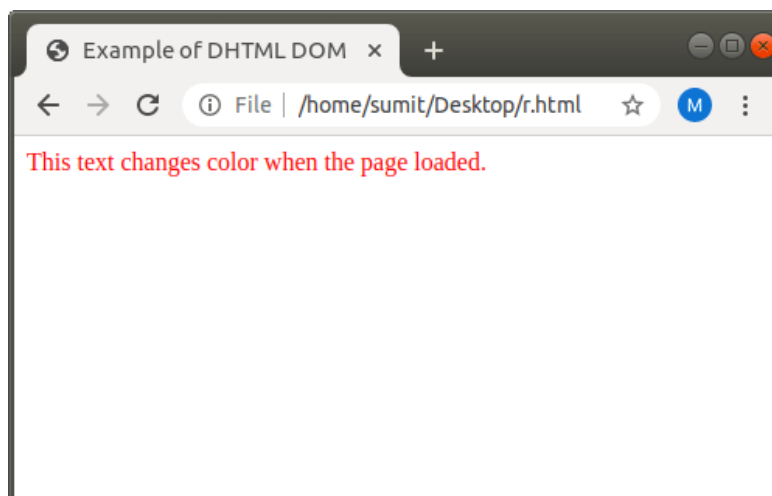
- **Dynamic Web Pages:** It allows you to create dynamic web pages. It enables the JavaScript to access and manipulate page content, structure, and style dynamically which gives interactive and responsive web experiences, such as updating content without reloading the entire page or responding to user actions instantly.
- **Interactivity:** With the DOM, you can respond to user actions (like clicks, inputs, or scrolls) and modify the web page accordingly.
- **Content Updates:** When you want to update the content without refreshing the entire page, the DOM enables targeted changes making the web applications more efficient and user-friendly.

- **Cross-Browser Compatibility:** Different browsers may render HTML and CSS in different ways. The DOM provides a standardized way to interact with page elements.
- **Single-Page Applications (SPAs):** Applications built with frameworks such as React or Angular, heavily rely on the DOM for efficient rendering and updating of content within a single HTML page without reloading the full page.

Example: The following program helps in understanding the concept of DHTML DOM. This example changes the color of text and displays the text in red colour automatically.

1. `<html>`
2. `<head>`
3. `<title>`
4. Example of DHTML DOM
5. `</title>`
6. `</head>`
7. `<body>`
8. `<font color = "blue">`
9. `<p id="demo"> This text changes color when the page loaded. </p>`
10. `</font>`
11. `<script type="text/javascript">`
12. `document.getElementById('demo').style.color = "red";`
13. `</script>`
14. `</body>`
15. `</html>`

Output:



Structure of DOM

DOM can be thought of as a Tree or Forest (more than one tree). The term **structure model** is sometimes used to describe the tree-like representation of a document.

Each branch of the tree ends in a node, and each node contains objects

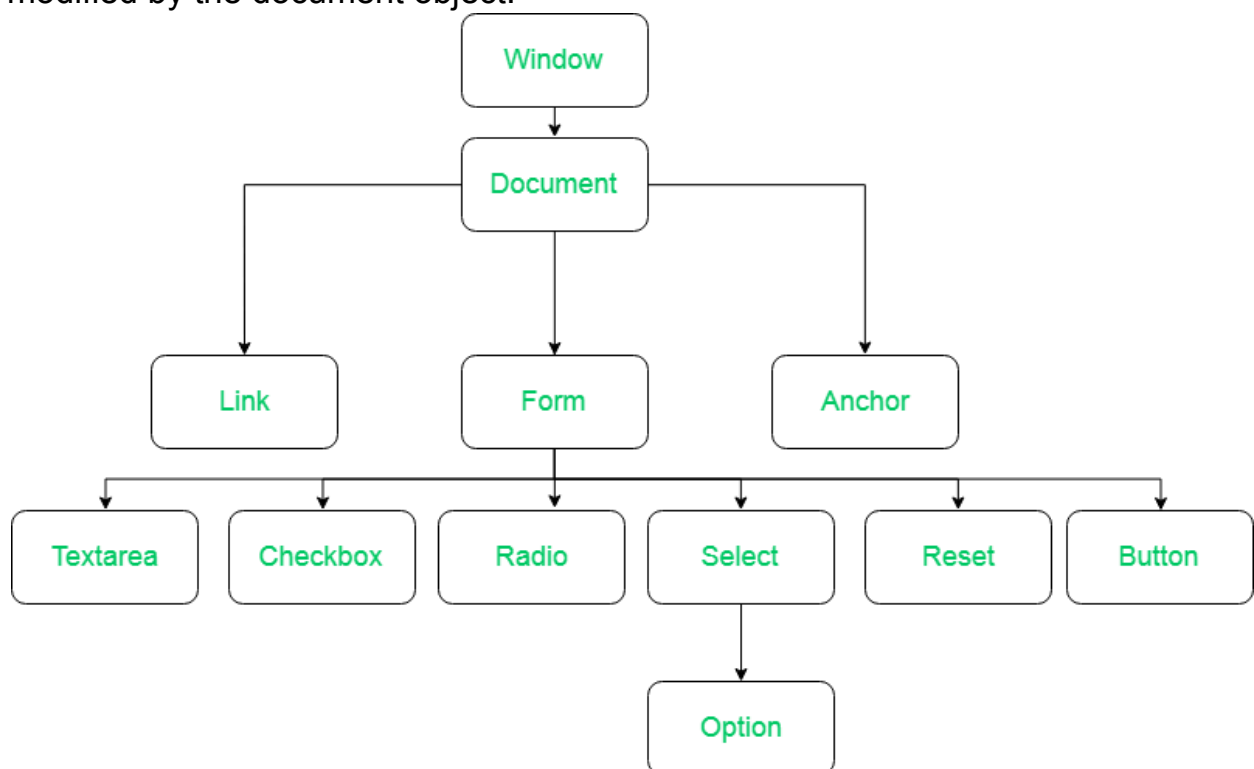
Event listeners can be added to nodes and triggered on an occurrence of a given event. One important property of DOM structure models is **structural isomorphism**: if any two DOM implementations are used to create a representation of the same document, they will create the same structure model, with precisely the same objects and relationships.

Why DOM is called an Object Model?

Documents are modeled using objects, and the model includes not only the structure of a document but also the behavior of a document and the objects of which it is composed like tag elements with attributes in HTML.

Properties of DOM

Let's see the properties of the document object that can be accessed and modified by the document object.



Representation of the DOM

- **Window Object**: Window Object is object of the browser which is always at top of the hierarchy. It is like an API that is used to set and access all the properties and methods of the browser. It is automatically created by the browser.
- **Document object**: When an HTML document is loaded into a window, it becomes a document object. The 'document' object has various properties that refer to other objects which allow access to and modification of the content of the web page. If there is a need to access any element in an HTML page, we always start with accessing the 'document' object. Document object is property of window object.
- **Form Object**: It is represented by ***form*** tags.
- **Link Object**: It is represented by ***link*** tags.
- **Anchor Object**: It is represented by ***a href*** tags.
- **Form Control Elements**: Form can have many control elements such as text fields, buttons, radio buttons, checkboxes, etc.

Methods of Document Object

DOM provides various methods that allows users to interact with and manipulate the document. Some commonly used DOM methods are:

- **write("string")**: Writes the given string on the document.
- **getElementById()**: returns the element having the given id value.
- **getElementsByName()**: returns all the elements having the given name value.
- **getElementsByTagName()**: returns all the elements having the given tag name.
- **getElementsByClassName()**: returns all the elements having the given class name.

Example

In this example, We use HTML element id to find the DOM HTML element.

• HTML

```
<!DOCTYPE html>

<html>

<body>
```

```

<h2>GeeksforGeeks</h2>

<!-- Finding the HTML Elements by their Id in DOM -->

<p id="intro">A Computer Science portal for geeks.</p>

<p>This example illustrates the <b>getElementById</b> method.</p>

<p id="demo"></p>

<script>

    const element = document.getElementById("intro");

    document.getElementById("demo").innerHTML =

        "GeeksforGeeks introduction is: " + element.innerHTML;

</script>

</body>

</html>

```

Output:

GeeksforGeeks

A Computer Science portal for geeks.

This example illustrates the **getElementById** method.

GeeksforGeeks introduction is: A Computer Science portal for geeks.

Getting the HTML element by `getElementById()` Method

Example

This example describes the representation of the HTML elements in the tree structure.

- html

```
<table>
```

```
  <ROWS>
```

```
    <tr>
```

```
      <td>Car</td>
```

```
      <td>Scooter</td>
```

```
    </tr>
```

```
    <tr>
```

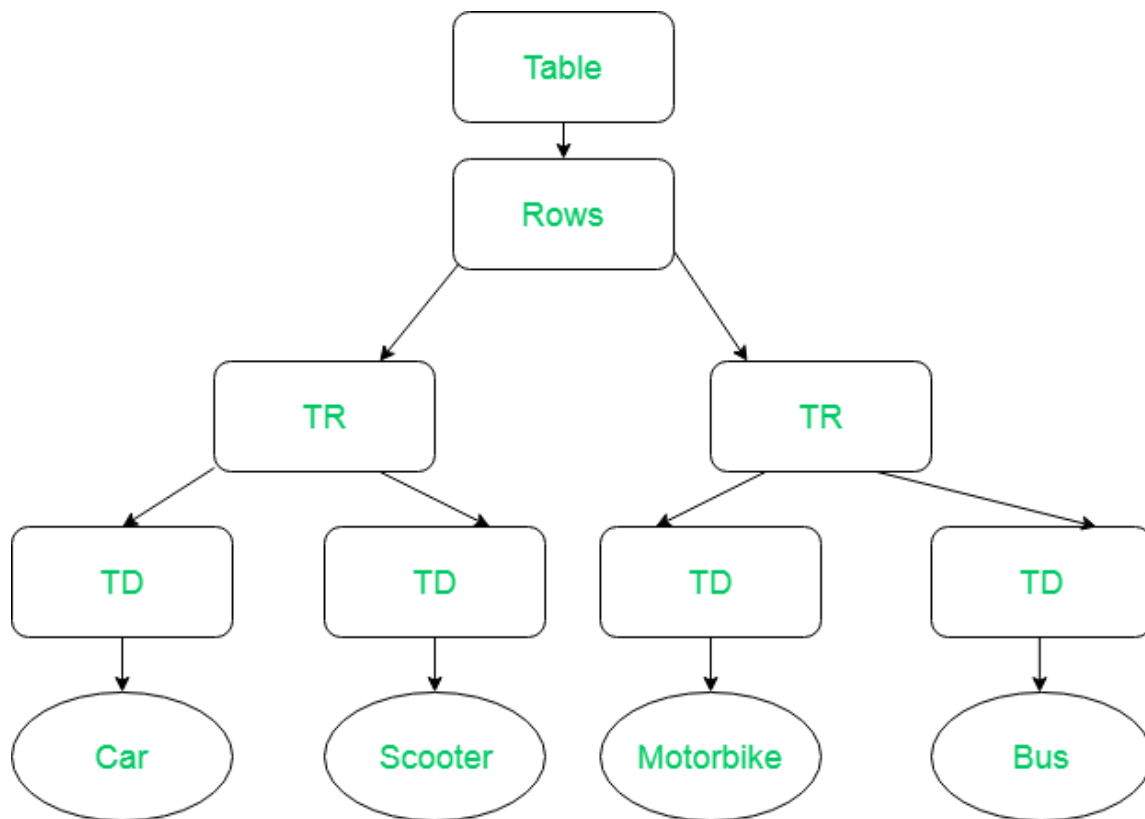
```
      <td>MotorBike</td>
```

```
      <td>Bus</td>
```

```
    </tr>
```

```
  </ROWS>
```

```
</table>
```



HTML elements in tree-like structure

What DOM is not?

- The Document Object Model is not a **binary description** where it does not define any binary source code in its interfaces.
- The Document Object Model is not used to describe **objects in XML or HTML** whereas the DOM describes XML and HTML documents as objects.
- The Document Object Model is not represented by a **set of data structures**; it is an interface that specifies object representation.
- The Document Object Model does not show the **criticality of objects** in documents i.e it doesn't have information about which object in the document is appropriate to the context and which is not.

Advantages of DHTML

Following are the various benefits or the advantages of DHTML (Dynamic HTML):

- o Those web sites and web pages which are created using this concept are fast.
- o There is no plug-in required for creating the web page dynamically.
- o Due to the low-bandwidth effect by the dynamic HTML, the web page functionality is enhanced.

- o This concept provides advanced functionalities than the static HTML.
- o It is highly flexible, and the user can make changes easily in the web pages.

Disadvantages of DHTML

Following are the various disadvantages or limitations of DHTML (Dynamic HTML):

- o The scripts of DHTML does not run properly in various web browsers. Or in simple words, we can say that various web browsers do not support the DHTML. It is only supported by the latest browsers.
- o The coding of those websites that are created using DHTML is long and complex.
- o For understanding the DHTML, users must know about HTML, CSS, and JavaScript. If any user does not know these languages, then it is a time-consuming and long process in itself.

CSSP(cascading style sheet positioning)

The **CSS position property** is used *to set position for an element*. it is also used to place an element behind another and also useful for scripted animation effect.

You can position an element using the top, bottom, left and right properties. These properties can be used only after position property is set first. A position element's computed position property is relative, absolute, fixed or sticky.

Let's have a look at following CSS positioning:

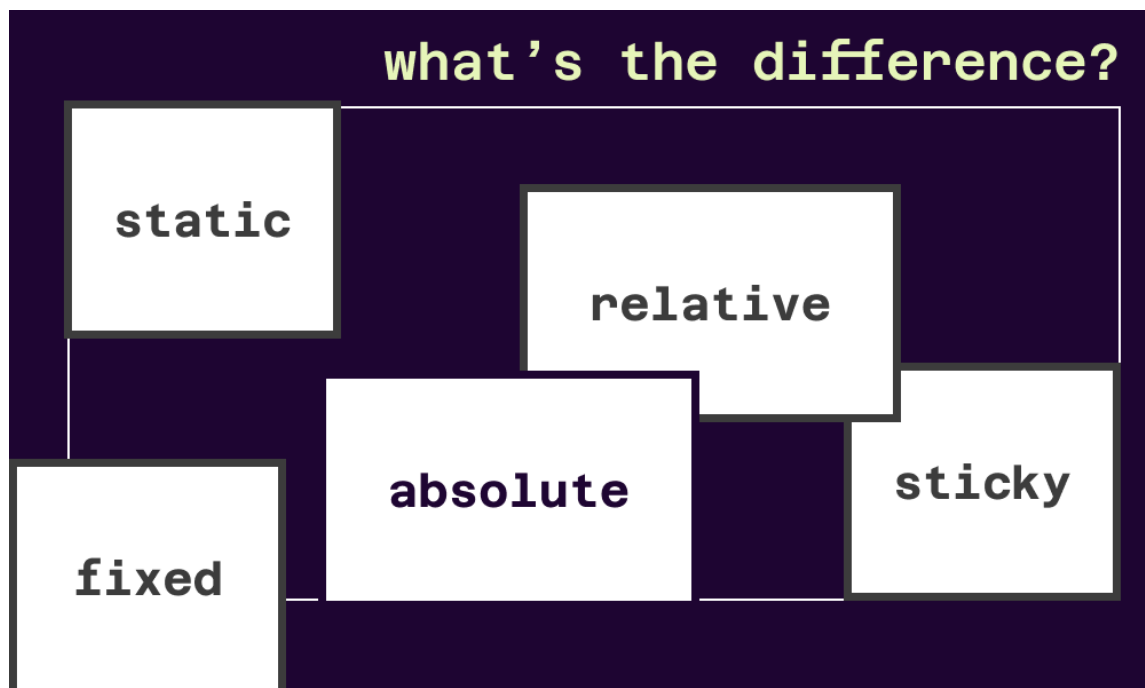
1. CSS Static Positioning
2. CSS Fixed Positioning
3. CSS Relative Positioning
4. CSS Absolute Positioning

1) CSS Static Positioning

This is a by default position for HTML elements. It always positions an element according to the normal flow of the page. It is not affected by the top, bottom, left and right properties.

static, relative, absolute, fixed, sticky - Understanding the difference

#css



CSS has different positions that can be applied to elements. They include `static`, `relative`, `absolute`, `fixed`, and `sticky`. These positions specify how an element should be positioned in a document which causes the element to behave differently.

In this article, we will understand the difference between these positions.

Positioned and Non-positioned elements

Before looking at these position styles, the first thing to understand is positioned and non-positioned elements.

Non-positioned elements appear on a page following the order they are declared in the DOM, and they occupy as much space as they need. Vertical and horizontal positioning (`top`, `left`, `right`, `bottom`) cannot be applied to such elements because they are not positioned.

In contrast, vertical and horizontal positioning can be applied to positioned elements. Positioned elements, by default, appear on a page like they are static, but using vertical and horizontal positioning can affect them differently, depending on the type of position.

Now, let's look at these position styles.

Static position

`static` position is the default position style for elements. With this style, elements are non-positioned--they appear as they are in the markup document. This style is also the only non-positioned style.

`top`, `left`, `right`, and `bottom` does not work with this style. You can visualize it with this code:

```
<div class="container">
  <div class="red-block"></div>
  <div class="blue-block"></div>
  <div class="green-block"></div>
</div>
.container {
  margin: 20px;
  height: 200px;
  display: flex;
  border: 1px solid black;
  padding: 20px;
  width: 400px;
}

.red-block,
.blue-block,
```



```

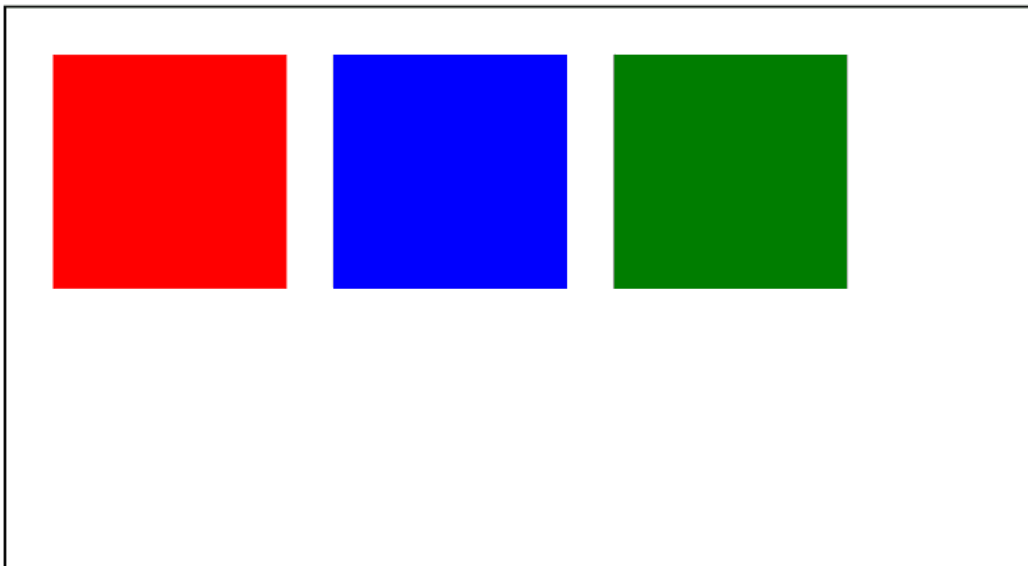
.green-block {
  width: 100px;
  height: 100px;
  margin-right: 20px;
}

.red-block {
  background-color: red;
}

.blue-block {
  background-color: blue;
  left: 20px;
  top: 20px;
}

.green-block {
  background-color: green;
}

```



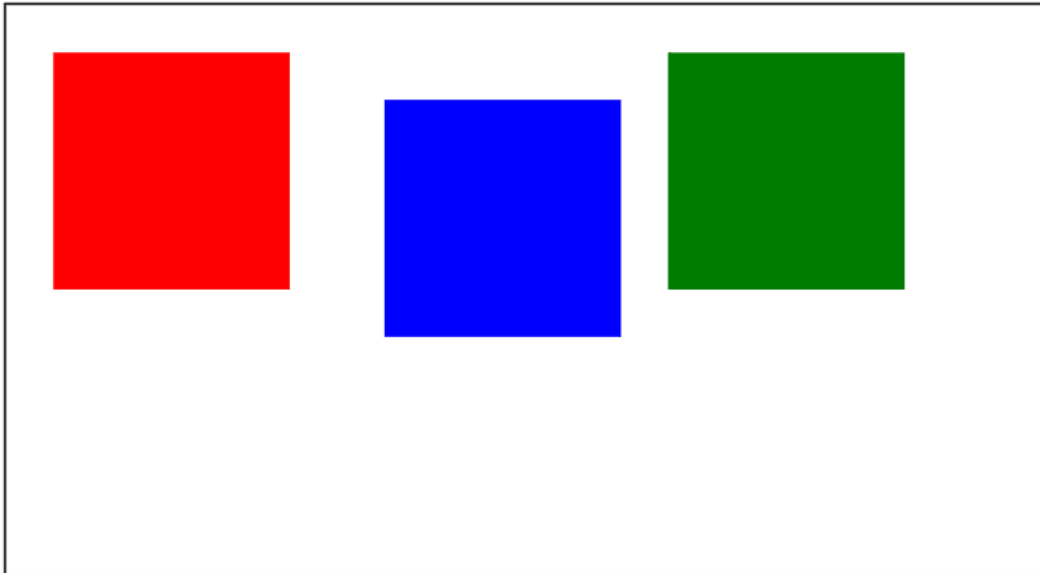
The `left` and `bottom` style declarations on the blue block are ignored. Of course, you can apply margins, but that would affect the element after it:

```

.blue-block {
  /* ... */
}

```

```
margin-left: 20px;  
margin-top: 20px;  
}
```

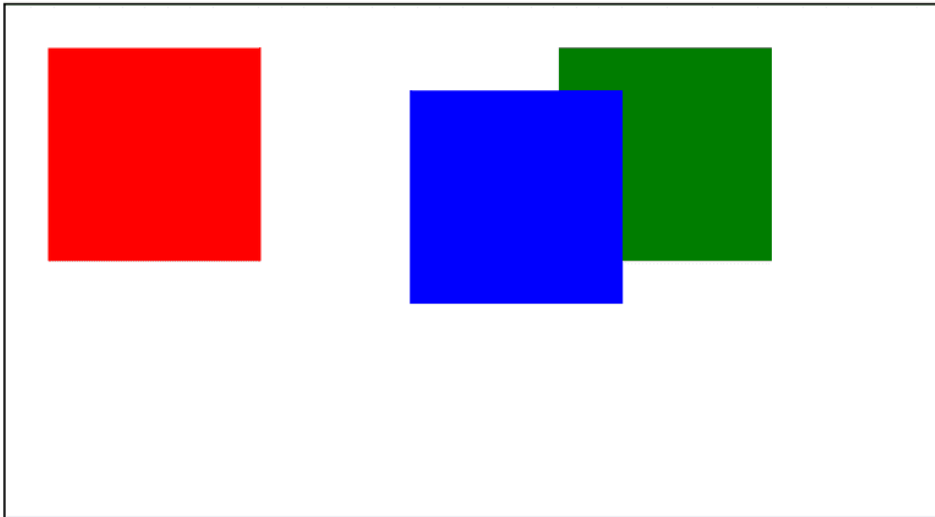


Relative position

You can think of the `relative` position as a style that gives `static` elements more flexibility. But unlike `static`, elements with the `relative` position are considered positioned elements. This means that such elements can appear differently from the markup flow.

With `relative`, the element retains its flow in the document and occupies as much space as needed by default, but you can use positioning properties like `top`. The idea here is that the element is relative to its default position. Using these positioning elements moves the element around the default position without affecting others. Here is what I mean:

```
.blue-block {  
  position: relative;  
  top: 20px;  
  left: 50px;  
}
```

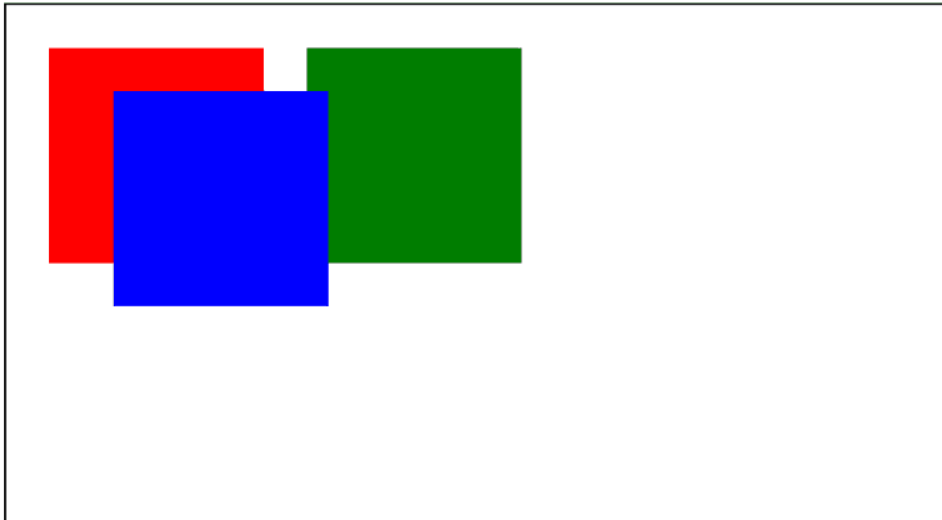


The blue block is still assumed to be filling up its default space, but positioning styles can move it around without pushing the others.

Absolute position

Elements with the `absolute` position are positioned elements that are removed from the flow of the document--like they are not there. Their space on the screen is taken away from them and assigned to other elements. Here's what I mean:

```
.blue-block {  
  /* */  
  position: absolute;  
  top: 40px;  
  left: 50px;  
}
```



As you would notice, the blue block's space earlier is now occupied by the green block. Even if you use margins on the absolute block, it does not affect the others. You can see it as the absolute element in its own territory outside the container.

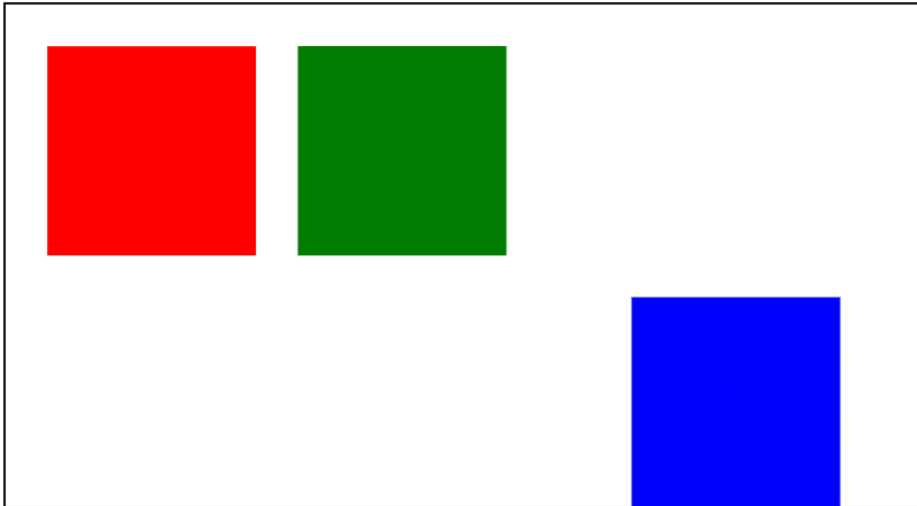
As it is currently, the absolute element has no relationship with the container. Absolute elements are positioned within the closest `relative` positioned parent, and if none, they are placed within the viewport (browser window). For example, with a positioning style of `left: 0`, the element moves to the left edge of the relative parent or the viewport.

To set a relationship between the blue block and the container, you need to apply a `relative` class to the container:

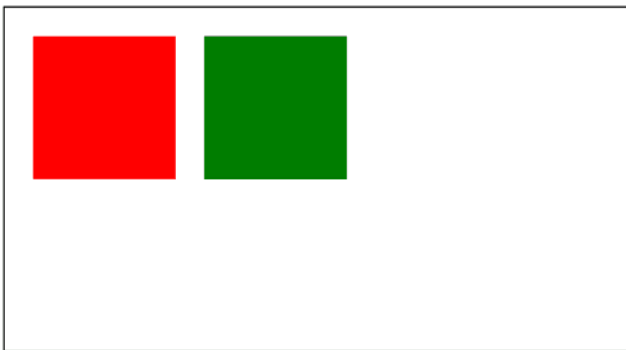
```
.container {  
  /* */  
  position: relative;  
}
```

Now, you can push the blue block to the edges of the container:

```
.blue-block {  
  /* */  
  bottom: 0;  
  right: 20px;  
}
```



As you can see, the container serves as a boundary that the positioning properties can move the absolute element along. If there were no closest `relative` parents, the blue block would appear like this:



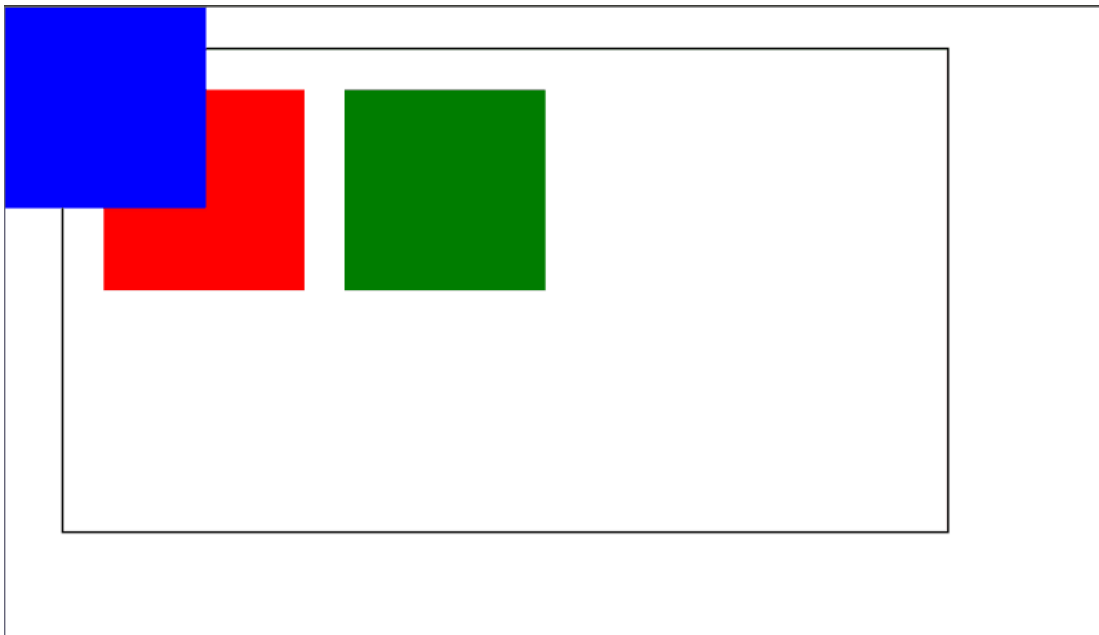
Fixed position

The `fixed` position is similar to the `absolute` position. The difference is that the fixed position does not respect any relative parents (or ancestors). It only respects the viewport. So a style like this:

```
.container {  
  /* */  
  position: relative;  
}
```

```
}  
  
.blue-block {  
  /* */  
  position: fixed;  
  top: 0;  
  left: 0;  
}
```

will produce this:



The `fixed` element only respects the viewport regardless of the relative parent.

Sticky position

As the name implies, this makes an element stick to a container. The `sticky` position toggles between the `relative` and `fixed` position in a scrolling container. An element of this position style starts with the `relative` position, retaining its flow in the document. Upon scrolling in the container, if the specified positioning distance (with `top`, for example) is met, the element becomes fixed until the scrolling container is out of view.

Here's an example to explain this:

```
<div class="container">
  <div class="red-block"></div>
  <div class="blue-block"></div>
  <div class="green-block"></div>
  <div class="other-block"></div>
</div>
.container {
  width: 400px;
  height: 200px;
  border: 1px solid black;
  overflow-y: auto;
  margin: 20px;
  padding: 20px;
}

.red-block,
.blue-block,
.green-block {
  width: 100px;
  height: 100px;
  margin-right: 20px;
}

.red-block {
  background-color: red;
}

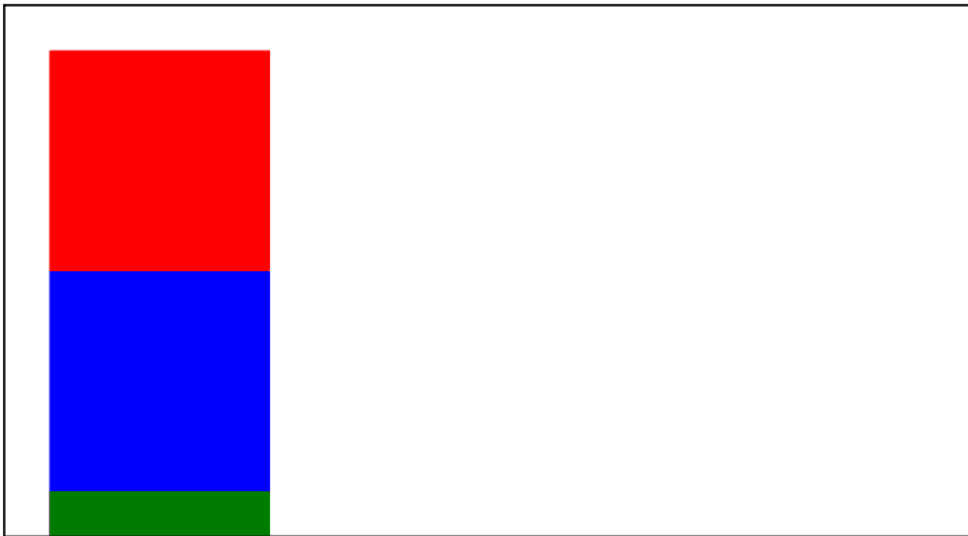
.blue-block {
  background-color: blue;
  position: sticky;
  top: 10px;
}

.green-block {
  background-color: green;
}

.other-block {
```

```
height: 500px;  
margin-top: 20px;  
width: 100%;  
background-color: rgb(61, 61, 61);  
}
```

At the start, the blue block appears in the flow of the document like this:



When you scroll in the container, and the blue block meets the `top: 10px` condition, it becomes fixed as you scroll:



You can try it on this [codepen](#). Try scrolling in the container and see the blue block fixed.

All CSS Position Properties

No	property	description	values
1)	bottom	It is used to set the bottom margin edge for a positioned box.	auto, length, %, inherit
2)	clip	It is used to clip an absolutely positioned element.	shape, auto, inherit
3)	cursor	It is used to specify the type of cursors to be displayed.	url, auto, crosshair, default, pointer, move, e-resize, ne-resize, nw-resize, n-resize, se-resize, sw-resize, s-resize, w-resize, text, wait, help
4)	left	It sets a left margin edge for a positioned box.	auto, length, %, inherit
5)	overflow	This property is used to define what happens if content overflow an element's box.	auto, hidden, scroll, visible, inherit
6)	position	It is used to specify the type of positioning for an element.	absolute, fixed, relative, static, inherit
7)	right	It is used to set a right margin edge for a positioned box.	auto, length, %, inherit

8)	top	It is used to set a top margin edge for a positioned box.	auto, length, %, inherit
9)	z-index	It is used to set stack order of an element.	number, auto, inherit

JavaScript Style Sheets

JavaScript Style Sheets (JSSS) was a stylesheet language technology proposed by Netscape Communications in 1996 to provide facilities for defining the presentation of webpages.^[1] It was an alternative to the Cascading Style Sheets (CSS) technology

JavaScript is the world's most popular programming language.

JavaScript is the programming language of the Web.

JavaScript is easy to learn.

This tutorial will teach you JavaScript from basic to advanced.

Why Study JavaScript?

JavaScript is one of the **3 languages** all web developers **must** learn:

1. **HTML** to define the content of web pages
2. **CSS** to specify the layout of web pages
3. **JavaScript** to program the behavior of web pages

This tutorial covers every version of JavaScript:

- The Original JavaScript ES1 ES2 ES3 (1997-1999)
- The First Main Revision ES5 (2009)
- The Second Revision ES6 (2015)
- The Yearly Additions (2016, 2017 ... 2021, 2022)

Much of a browser's work is manipulating the display, and much of their display code already had been exposed for JavaScripting. So it seemed only natural, perhaps even relatively easy, for the developers at Netscape to implement JavaScript Style Sheets. this alternative document style technology lets you prescribe display properties for all the various HTML elements, either inline as tag attributes, at the document level, or for an entire document collection.

JavaScript Style Sheets (JSS) are a Netscape invention. In fact, for a short time in the fall of 1996, Netscape appeared ready to eschew the CSS methodology, which Internet Explorer already had implemented, and use JSS exclusively for HTML document designers with their new browser, Netscape Navigator 4.0. Fortunately, the new version now supports both JSS and CSS technologies.

JSS also is known as JavaScript Style Sheets is a combination of JS and CSS. It is a tool which uses JS to describe styles declaratively, which makes it conflict-free and more reusable.

When the javascript is parsed, CSS is generated attached to the DOM(usually gives style element i.e.,<style>)

From SASS to CSS-in-JS we resolved many style conflicts due to the CSS selectors, and complicated nesting of CSS selectors. JSS avoids such conflicts by generating unique class names when JS is parsed into CSS

Features Of JSS:

Real CSS:

JSS creates real CSS, not Inline Styles. It bolsters each current CSS highlight. CSS rules are made once and reused over the components utilizing its class name in opposition to Inline Styles.

Reusable styles:

Utilizing JavaScript as a host language offers us a chance to reuse CSS decides such that is beyond the imagination with ordinary CSS. You can use JavaScript modules, factors, capacities, math activities, and the sky is the limit from there. Whenever done right, it can, in any case, be completely definitive.

Conflict-free CSS:

JSS creates unique class names by default. It permits keeping away from the run of the mill CSS issue, where everything is worldwide as a matter of course. It totally evacuates the requirement for naming shows.

Dynamic CSS:

Utilizing JavaScript capacities and Observables makes it conceivable to progressively create styles in the program, allowing you a chance to get to your application state, program APIs or remote information for styling. You can characterize styles once as well as update them anytime in an effective manner.

Critical CSS:

To upgrade time to initially paint, you can utilize server-side rendering and concentrate basic CSS. You can couple the rendering of CSS with the rendering of HTML so that no unused CSS gets created. It will result in a negligible basic CSS removed amid server-side rendering and enable you to inline it.

Expressive syntax of CSS:

Because of different modules, JSS enables you to have settling, worldwide selectors, an organization with existing worldwide class names. For example, JSS-module extend enables you to express properties like box-shadow even in a more coherent manner than it is conceivable with CSS. You can likewise utilize layout strings on the off chance that you need to duplicate glue styles legitimately from the program dev instruments.

The simplicity of removal and modification:

Express use of CSS rules enables you to follow them down to the purchaser and choose on the off chance that it very well may be securely evacuated or altered.

JavaScript fabricate pipeline:

There is no requirement for extra form pipeline arrangement for CSS. Whatever apparatus you manufacture your JavaScript, it will simply work with JSS.

A comparison of CSS and JSS below

CSS:

```
var font = {  
  fontSize: 12,  
  lineHeight: 1  
};  
  
export default {  
  button: {  
    extend: font,  
    border: 'none',  
    margin: [5, 10],  
    transition: ['background', 'color', 'font-size'],  
    transitionDuration: 300,  
    background: {  
      color: 'white',  
      image: 'url("/some/url/image.png")',  
      repeat: 'no-repeat',  
      position: 'contain'  
    },  
    '&:before': {
```

```
    content: "icon"
  }
},
redButton: {
  extend: 'button',
  background: 'linear-gradient(to right, red 0%, green 100%)',
  fallbacks: {
    background: 'red'
  },
  '&:hover': {
    border: [
      [1, 'solid', 'blue']
    ],
    boxShadow: [
      [0, 0, 0, 10, 'blue'],
      [0, 0, 0, 15, 'green']
    ],
    '& span': {
      color: 'red'
    }
  }
},
'@media (min-width: 1024px)': {
  button: {
    fontSize: 16
```

```
}  
  
}  
};
```

JSS:

```
.button-0-1-7 {  
  border: none;  
  margin: 5px 10px;  
  font-size: 12px;  
  transition: background color font-size;  
  background: white contain no-repeat;  
  line-height: 1;  
  background-image: url("/some/url/image.png");  
  transition-duration: 300ms;  
}  
  
.button-0-1-7:before {  
  content: "icon";  
}  
  
.redButton-0-1-8 {  
  background: red;  
  border: none;  
  margin: 5px 10px;  
  font-size: 12px;  
  transition: background color font-size;  
  background: linear-gradient(to right, red 0%, green 100%);
```

```
line-height: 1;
transition-duration: 300ms;
}
.redButton-0-1-8:before {
  content: "icon";
}
.redButton-0-1-8:hover {
  border: 1px solid blue;
  box-shadow: 0 0 0 10px blue, 0 0 0 15px green;
}
.redButton-0-1-8:hover span {
  color: red;
}
@media (min-width: 1024px) {
  .button-0-1-7 {
    font-size: 16px;
  }
}
```

JSS is less code with conflict-free classes where the CSS generated from JS is not inline and can be used globally. Easily readable and a declarative approach to writing styles.

JavaScript in <head>

In this example, a JavaScript function is placed in the <head> section of an HTML page.

The function is invoked (called) when a button is clicked:

Example

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>

<h2>Demo JavaScript in Head</h2>

<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>

</body>
</html>
```

OUTPUT

Demo JavaScript in Head

A Paragraph.

Try it

Paragraph changed.

Types of JavaScript Operators

There are different types of JavaScript operators:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- String Operators
- Logical Operators
- Bitwise Operators
- Ternary Operators
- Type Operators
-

JavaScript Operators

The **Addition Operator** + adds numbers:

The **Assignment Operator** = assigns a value to a variable.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Operators</h1>
```

```
<h2>The Assignment (=) Operator</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
// Assign the value 5 to x
```

```
let x = 5;
```

```
// Assign the value 2 to y
```

```
let y = 2;
```

```
// Assign the value x + y to z
```

```
let z = x + y;
```

```
// Display z
```

```
document.getElementById("demo").innerHTML = "The sum of x + y is: "  
+ z;
```

```
</script>
```

```
</body>
```

```
</html>
```

OUTPUT

JavaScript Operators

The Assignment (=) Operator

The sum of x + y is: 7

JavaScript Functions

A JavaScript function is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

```
// Function to compute the product of p1 and p2  
function myFunction(p1, p2) {  
  return p1 * p2;  
}
```

Exp

```
<!DOCTYPE html>

<html>

<body>

<h1>JavaScript Functions</h1>

<p>Call a function which performs a calculation and returns the
result:</p>

<p id="demo"></p>

<script>

function myFunction(p1, p2) {

    return p1 * p2;

}

let result = myFunction(4, 3);

document.getElementById("demo").innerHTML = result;

</script>

</body>

</html>
```

OUTPUT

JavaScript Functions

Call a function which performs a calculation and returns the result:

12

Exp

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>My First JavaScript</h2>
```

```
<button type="button"
```

```
onclick="document.getElementById('demo').innerHTML = Date()">
```

```
Click me to display Date and Time.</button>
```

```
<p id="demo"></p>
```

```
</body>
```

```
</html>
```

OUTPUT

My First JavaScript

Click me to display Date and Time.

Thu Jan 11 2024 10:16:48 GMT+0530 (India Standard Time)

Use the Menu

We recommend reading this tutorial, in the sequence listed in the menu.

If you have a large screen, the menu will always be present on the left.

If you have a small screen, open the menu by clicking the top menu sign .

JavaScript Can Change HTML Content

One of many JavaScript HTML methods is `getElementById()`.

The example below "finds" an HTML element (with `id="demo"`), and changes the element content (`innerHTML`) to "Hello JavaScript":

```
document.getElementById("demo").innerHTML = "Hello JavaScript";
```

EXP2

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p id="demo">JavaScript can change HTML content.</p>
```

```
<button type="button"
onclick='document.getElementById("demo").innerHTML = "Hello
JavaScript!'">Click Me!</button>
```

```
</body>
```

```
</html>
```

An Introduction to Netscape Layers

- **DHTML**
- Layers
- Inline Layers
- Embedding HTML files

DHTML

Welcome to the world of DHTML otherwise know as Dynamic HTML. If you've never learned HTML now would be a great time to learn because every place you go is using some sort of HTML for use on the Internet or Intranet. HTML can become the fastest way to communicate and change data fast with out having to spend a fortune to maintain it.

But how do you go about attracting people? Well HTML can only go so far. What would happen for example if you could make a web site with moving images and text that changes while not effecting the speed of your users connection? You would end up making a better web site which would change the minds of many.

I hope to explain and give examples of small things in which you can make your web site so much better.

So you've heard that DHTML is so great but what are the cons? Well unfortunately there are cons to DHTML. But there are cons to JavaScript. There are two versions of DHTML. One for Netscape Navigator. The other in Microsoft Internet Explorer.

For some reason Microsoft and Netscape decided that they needed versions which wouldn't run on all browsers and that when using DTHML you would have to use different HTML tags and syntax to get both browsers to work.

The code in this article will only work with Netscape Navigator 4.

Layers

The first example is how to setup two layers of content. This way seems odd at first but read on and you will understand. So with no more delay I will get started.

The first step is to setup a base line in which your browser will talk to the HTML document. The next step will be to make the layer. To do that follow the example below.

```
<LAYER>DHTML</LAYER>
```

OK great so you've just lost me. Why did I just write two sets of different layers? The reason you've written two different layers is because it's like the feathers of a bird. If a bird only had one feather then how would it look? But in real life the bird has several layers. So that's what you've just done. By creating several layers you can make your site even more interesting!

So this is what the code is going to look like when you are done:

```
<LAYER><FONT COLOR="#FF0000">DHTML</FONT></LAYER>  
<LAYER><FONT COLOR="#0000FF">Rules</FONT></LAYER>
```

Here is the working example:

DHTML Rules

Didn't that look interesting? See how the words are layered on top of each other. If not make sure you are running Netscape Navigator 4 or later.

The problem with what we've just done is that the words lay on top of each other and for your home page that's just not going to work. So lets take our existing code and change it a little so everything will look a bit better. This time lets add something to our lines.

```
<LAYER><FONT COLOR="#FF0000">This is an example of layering  
with Netscape Navigator 4</FONT></LAYER>  
<LAYER><FONT COLOR="#0000FF"><CENTER>DHTML  
Rules</CENTER></FONT></LAYER>
```

Here is the working example:

This is an example of layering with Netscape Navigator 4

DHTML Rules

See how "DHTML Rules!!!" is centered across the whole page, and at the same time to the right of "This is an example of..."? But still I'm just not happy with this. I want my site to be the best on the intranet!

Inline Layers

```
<ILAYER><FONT COLOR="#FF0000" SIZE="+2">DHTML</FONT></ILAYER>  
<LAYER><FONT COLOR="#0000FF" SIZE="+4">Rules</FONT></LAYER>
```

Here is the working example:

DHTML Rules

This example has introduced two new HTML tags, the `<ILAYER>` and `</ILAYER>` tags. What I didn't explain in the previous examples is the Netscape LAYER tag opens a layer that is independent of the rest of the HTML around it, i.e. it does not reserve space in the normal document flow. Whereas the Netscape ILAYER places the layer inline with the surrounding HTML. So in the above example the two layers do not overlap because the first layer has reserved just enough space within the document for itself, the second layer then just follows on from any preceeding HTML.

The next example attempts to demonstrate this in more detail:

```
<ILAYER><FONT COLOR="#FF0000" SIZE="+2">DHTML</FONT></ILAYER>  
<LAYER><FONT COLOR="#0000FF" SIZE="+4">Rules</FONT></LAYER>  
<LAYER><FONT COLOR="#00FF00" SIZE="+6">Okay?</FONT></LAYER>
```

HTML id Attribute

The HTML id attribute is used to specify a unique id for an HTML element.

You cannot have more than one element with the same id in an HTML document.

Using The id Attribute

The id attribute specifies a unique id for an HTML element. The value of the id attribute must be unique within the HTML document.

The id attribute is used to point to a specific style declaration in a style sheet. It is also used by JavaScript to access and manipulate the element with the specific id.

The syntax for id is: write a hash character (#), followed by an id name. Then, define the CSS properties within curly braces {}.

In the following example we have an <h1> element that points to the id name "myHeader". This <h1> element will be styled according to the #myHeader style definition in the head section:

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
#myHeader {
    background-color: lightblue;
    color: black;
    padding: 40px;
    text-align: center;
}
```

```
</style>
</head>
<body>

<h1 id="myHeader">My Header</h1>

</body>
</html>
```

OUTPUT

The id Attribute

Use CSS to style an element with the id "myHeader":

My Header

Difference Between Class and ID

A class name can be used by multiple HTML elements, while an id name must only be used by one HTML element within the page:

Example

```
<style>
/* Style the element with the id "myHeader" */
#myHeader {
    background-color: lightblue;
    color: black;
    padding: 40px;
    text-align: center;
}

/* Style all elements with the class name "city" */
.city {
    background-color: tomato;
    color: white;
```

```
padding: 10px;
}
</style>

<!-- An element with a unique id -->
<h1 id="myHeader">My Cities</h1>

<!-- Multiple elements with same class -->
<h2 class="city">London</h2>
<p>London is the capital of England.</p>

<h2 class="city">Paris</h2>
<p>Paris is the capital of France.</p>

<h2 class="city">Tokyo</h2>
<p>Tokyo is the capital of Japan.</p>
```

OUTPUT

Difference Between Class and ID

A class name can be used by multiple HTML elements, while an id name must only be used by one HTML element within the page:

My Cities

London

London is the capital of England.

Paris

Paris is the capital of France.

Tokyo

Tokyo is the capital of Japan.

CHAPTER 13

DYNAMIC HTML (DHTML)

13.1 Introduction

Those users like to browse the web sites which are more attractive, dynamic and eye-catching. While creating a HTML page you had found some limitations but DHTML overcomes limitations and fulfills the requirement.

DHTML allow the web authors to add interactive and dynamic features in the web, which makes the web more attractive.

13.2 DHTML

What is DHTML? Most web developers don't actually know, or only have a vague idea of what the term means; a consequence of the fact that no official specs or definition have ever been issued. The accepted definition for this new item in the nomenclature is the **changing of the style declarations of a HTML element through JavaScript.**

When you execute a change to the style declaration of an element, the change is registered immediately in the browser window, without the page having to be refreshed. You can have a reader interacting with a single page, hiding and showing elements, moving things around, changing colours and generally messing everything about — and never have to call for further files or load new pages.

Before the advent of DHTML, we could only use static HTML. When you load up a page, aside from the odd JavaScript rollover, the view didn't change in appearance until you moved on to the next page. Elements were placed onto the page in a natural flow according to the order they assume in the HTML code. DHTML, being dynamic and all, allows you to disrupt this natural flow by taking elements out of the page and displaying them elsewhere, on command from user interactions. Clicking on links and buttons can allow all sorts of things to take place. The order things come in the source code can be made largely irrelevant with absolute positioning.

13.3 Advantages of DHTML

The Various Advantages of DHTML are :

1. Creates interactive web pages
2. It removes the load of processing data, picture or may be an animation from the web server to the client machine.
3. It can be embedded in a HTML file without creating a separate file.
4. It is a combination of HTML, Scripting and object-oriented programming which creates beautiful web pages.
5. Dynamic HTML document displays on its own structure known as DOM (Document Object Model).

13.4 How does DOM Works?

While working with HTML files you have seen that HTML deals with the structure of the page. You can create paragraphs, headings, load picture, put background, create tables, frames etc. So you have displayed them as it appears. For example, you have written a paragraph just by using the <P> tags and ended it with </P> tag. You haven't created a paragraph with different color, margin, indent etc. from other paragraphs. So these are the limitations with HTML, but now with DHTML you can create the structure of the document with the help of DOM (Document Object Model).

You are embedding the styles in the same page (i.e. in the same text file) but DOM works in a different way. Here the browser, identifies each element differently - where the element is placed, its attributes etc. In this way it can keep a track of each element of the browser (like headings, paragraphs, image etc). The browser makes a stack of similar objects after identifying them. For example You have put five headings so the browser creates a database and stores the record of the headings each time the pages gets uploaded.

Netscape is the first company to develop DOM Netscape Navigator 2.0. Later in Microsoft Internet Explorer DOM gets implemented.

At the beginning DOM was described as an Instance hierarchy of JavaScript object.

Instance. The object will not be created until and unless they appear on the web page. For example, The link object will be empty in the absence of hyperlink.

Hierarchy. Each object created sets a relation with other objects. Follow the chain where you will find that the Browser sets relation with the window object and window object sets relation with document.

Again if you notice you will find that document has three objects links, anchors and form. The Form had a chain of form elements. So overall you can say that it maintains a descending hierarchy.

13.5 Features of DHTML

The various features of DHTML are as follows :

1. **Changing the tags.** With the use of DOM you can change the tags and the matter of the style sheets.

2. **Dynamic Fonts.** In HTML, you have mentioned a font name and when you have viewed it in the browser it appears accordingly. But think of it when you are creating live projects, you are assigning a font which may not be available in the users machine. So you don't know how the font will appear after loading in the user machine.

To overcome this problem Netscape developed the idea of dynamic fonts. So with the font tag we are going to put a series of fonts according to the preference followed by a general type.

3. **Positioning an Elements.** When you have worked with HTML you couldn't position an element. Whereas with DHTML you can position an element according to your choice. If you are using Microsoft Internet Explorer it uses the style sheets <DIV> tags and if you are using Netscape Navigator you can use <DIV> tags and the <LAYER> tags.

4. **Data Binding.** You can use any database like SQL, ODBC and JDBC but you require an advance programming language for accessing a database. You can use any-language as per your knowledge. Now a days, Perl is used for Database handling which access the network and delivers required information.

5. **The World Wide Consortium (W3C).** The World Wide Consortium (W3C) is a recognized organization develops common standards. It also, updates HTML and HTTP standards, mark-up Languages accessibility, related issues and browser implementations.

Microsoft and Netscape both were aware about the look of DHTML. Microsoft calls its implementation with capital 'D' and Netscape calls its implementation with lowercase 'd'.

The World Wide Consortium (W3C) has developed its own recommendations for DHTML. The different capabilities are listed below.

The Key components of the W3C DOM are universal. So that the user can use it in any browser and on any computer i.e. there is no limitations on computer configuration also.

DOM should be language independent. W3C wanted to separate DOM from different languages. The properties of the DOM can be used on any language without changing its structure.

DOM is accessible for all users. A graphical browser can work with text, images and all other visual manipulations ranging from heading styles, tables etc.

There are some non-graphical browsers like lynx. Different browsers have some limitations too.

W3C maintains the standard that promote HTML and DOM accessibility for people.

The last requirement is security, privacy and validity.

6. DOM Capabilities

Navigation Structure. This refers regarding navigation between the documents. Finding a parent element or a child element.

Manipulation of Content. This refers to manipulate the content and document. At any portion you can change the script or the content. For example, you can change H1 to H2, you may change the color etc.

Event Model. Event model makes the page interactive. This includes a wide variety of possible activities like mouse movements, mouse clicks, key press etc.

13.6 Cascading Style Sheets

Style sheets are the way that standards-compliant Web designers define the layout, look-and-feel, and design of their pages. They are called Cascading Style Sheets or CSS. With style sheets, a designer can define many aspects of a Web page:

- fonts
- colors
- layout
- positioning
- imagery
- accessibility

Style sheets give you a lot of power to define how your pages will look. And another great thing about them is that style sheets make it really easy to update your pages when you want to make a new design. Simply load in a new style sheet onto your pages and you're done.

13.6.1. CASCADING STYLE SHEETS (CSS)

HTML has its limitations when it comes to layout. CSS helps us achieve such layouts. With CSS, you have much better control over the layout of your web pages. You can specify exactly how big a font will be, exactly where an element will be on a page, what the page will look like when printed, and much more.

CSS can also save you a lot of time, particularly when maintaining a large site. Also, the Word Wide Web Consortium (W3C) recommends that web developers use CSS tags instead of HTML tags wherever possible. The W3C are gradually phasing out quite a few of the HTML tags.

CSS is a form of communication used to dilate the display of a website markup language like HTML or XHTML as it determines such features as its fonts, layouts, spacing and colors, to name just a few. Basically, CSS is used to take the content you already have and make it look more attractive; it is a language used to increase expressive style and creativity.

One of the favored features is its ability to allow the sorting of document content written in markup languages (like HTML) from document presentation written in CSS.

13.6.2. IMPLEMENTATION OF CSS

CSS files are termed "cascading" style sheets because of two reasons: one stylesheet can cascade, or have influence over multiple pages. Similarly, many CSS files can define a single page.

There are 3 ways to implement CSS commands into your site:

1. Use one CSS file for all your pages. This is the best way to do it.
2. Integrate CSS commands into the head of each of your documents.
3. Use the style attribute to put CSS code directly into a HTML element.

CSS allows you to use all three of these methods in glorious tandem, inheriting and overriding values as you go.

1. ONE CENTRAL STYLESHEET :

This is how you should use most of your CSS. You write just one css file and have all your pages referencing it. This way, a change to anything in this one file will adjust this thing (a font, for example) across your whole site. You could change your entire colour scheme with one modification if you want, over an unlimited number of pages. That's one of the things CSS was designed for—flexibility.

To create your stylesheet, open a text editor (Notepad will be fine). Remember earlier, you learned how to save from a text editor into the .html file format? Well, here you'll be doing roughly the same except your file will have a css suffix. Just save a blank file as mystyles.css and put it in the same directory as your homepage. Now that you have that, we can show you the syntax used in CSS:

selector {property : value; property : value; property : value;}

This is the basic syntax to your css file.

- The "Selector" is the label you give to your code. This label is added as a tag where you want the properties to be applied. It is usually the name of a tag, without its triangle brackets.
- The brackets are {curvy}, not [square] or (round).
- The "Property" are things like size, colour, font, etc.
- After the property name there is a colon, and between each individual part there is a semicolon. Each of these pairs of properties and values is a declaration.

Here is a working example:

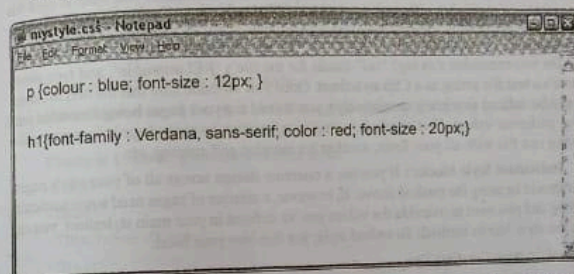
```
p {colour: blue; font-size: 12px;}
```

Just put that one line into your file. That's all you need. This rule applies to all paragraph elements. Once you've linked the stylesheet to your pages, having this style declaration in your css file would make all the text in your pages enclosed in the <p> and </p> tags.

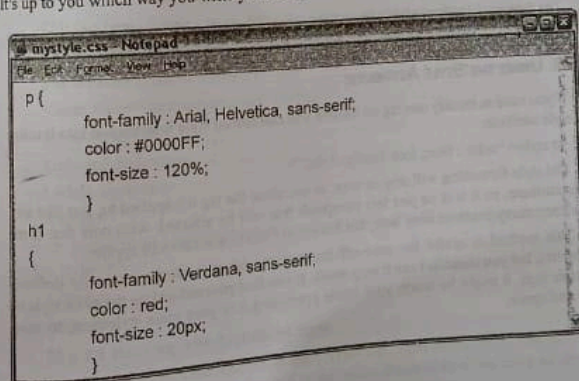
You could add another line in under your first one. Try this line of CSS and then use some large headings on your page:

```
h1 {font-family: Verdana, sans-serif; color: red; font-size: 20px; }
```

Your stylesheet should look something like this.



Another style of CSS coding which you can use is below. Many people prefer to use this; especially on commercial projects where someone else may have to read your code. It's up to you which way you write your CSS:



If you want to affect multiple selectors with the same CSS formatting, add them all together, with commas:

You can also add comments to your stylesheet using the `/*...*/` delimiters. These can give you or anyone else using your stylesheet vital tips about what's going on in your code.

2. ATTACHING YOUR STYLESHEET :

Now that you have something in your file, you'll need to show your pages where their css file is. Put this line of code into the head part of any documents you want to read this file:

```
<link rel="stylesheet" type="text/css" href="mystyles.css">
```

Do you remember this tag? "rel" stands for the file's 'RELationship', and type shows that it's a text file acting as a CSS stylesheet. Once you've made sure that the href is correct (it can be defined absolutely or relatively), you should see your pages being formatted with your preferred values. You can link multiple stylesheets to each page if you want, like having one file with all your fonts, another for margins and spacing, etc.

Individual Style blocks : If you use a common design across all of your site's pages, you should be using the method above. If, however, a number of pages need some particular styling and you need to override the values you've defined in your main stylesheet, you can use the style blocks method. To embed style, put this into your head:

```
<style type="text/css">
  p {font-weight: normal; color: gray;}
  h1 {color: black;}
</style>
```

The type attribute here allows browsers to treat this code as CSS. CSS code applied in this way is not technically a stylesheet, but is called an "inline style block."

3. USING THE STYLE ATTRIBUTE:

If you need to modify one tag on its own you can embed style information into it using the style attribute:

```
<p style="color: blue; font-family: Arial;">
```

The style formatting will stop as soon as you close the tag it's applied to, just like any other attribute, so it will be just this paragraph that will be affected. Also note that there aren't any curly brackets used here, but the colon / semicolon rule still applies.

This method is useful for once-off formatting, and overriding previously defined properties, but you shouldn't use it very much. If you find yourself adding the same style to multiple tags, it might be worth your while promoting it to your main stylesheet, to save time and space.

13.6.3. DIFFERENT WAYS TO ATTACH STYLE SHEETS TO TAG

The different ways to attach Style Sheets to tags are.

Inline Style Sheets. It adds a specific style to the document controlled by the tags. For ex. You want to color a paragraph with red. Then mention it as red and control the paragraph settings inside the span tags.

Embedded Style Sheets. This method embeds the styles inside the <Style> tags and ends with </Style> tags. They are placed inside the <Body> tags and inside.

External (Linked) Style Sheets. This method embeds the style from an external file. For ex. Style is created and saved with an extension css. Later it is linked with the HTML document.

13.6.3.1 Working with Inline Style

Here you have to set the style with the tags. Suppose you want to set the font style of the paragraph you should write <P style = "font style: Italic">

Example 1. his program shows Inline style.

```
<Html>
<Head>
<Title>Inline</Title>
</Head>
<Body bgcolor="lime">
<center><H1 Style="background-color:blue" id="h1">Inline Style</H1></Center>
<P Style="font-size:14pt" "4font-weight:bold" "font-style:italic" id="p1">
This is a type of style setting. Here the fonts size of the paragraph is 14pt.
The font appears bold and the font style appears in italic. </P>
The different types of styles are <Br>
<Span Style="font-weight:bold" id="s1">
Inline Style<Br>
Embedded Style<Br>
Linked Style<Br>
</Span>
```

<P style="font-size=16pt" "Color:yellow" id="p2"> This example describes the inline style</P>

```
</Body>
```

```
</Html>
```

13.6.3.2 Working with Embedded Style

Here you have to set the style inside the <Style> tags. First create the HTML document. Put the style in the <Style> tags in the editor. Remember if you have set the style

for H2 and while creating the HTML document you are not using the H2 then it is of no use. Create the HTML document with the tags for which you had set the styles.

Example 2. This program shows the Embedded Style

```
<Html>
<Head>
<Title>Embedded</Title>
</Head>
<STYLE>
BODY {background-color:lightblue; colour:red; margin-left:0.5in; margin-right:0.5in}
H1 {background-color:yellow; color:red}
H2 {color:blue}
P {font-size: 14pt; color:green;}
</STYLE>
</Head>
<Body>
<Br><Br>
<H1><center>Vision 2000</H1>
<H2>Computer Education</H2></Center>
<P>Authorised software user of Microsoft. We are providing education of various
Microsoft Software.
We are having ATC from FACT. Other than that we teach different subjects like DTP i.e
Desktop Publishing. Further we have 3yrs Diploma course, E-commerce course etc. </P>
</Body>
</Html>
```

13.6.3.3 Working with Link Style

Link style sheets are quite helpful. You can link it to any files according to your choice. Still now you have seen that you have to create the style each time you create a page. But with Link style you can create the style in a different file and then just link it with the HTML document.

What you have to do? Open, the editor for ex. Notepad. Then start from the <Style> tag instead of <HTML> tag. Now set the styles for each element as you have done in the previous example. Later when you finish save it with an extension of css. For ex. Style1.

Next part is to create a HTML document separately and then, set the link to this file. For ex. <link rel=stylesheet href= "Style1. css" type= "text/css">

In the same way you could have linked a JavaScript Style sheets too. Then the type would change to text/js.

For example :

```
<STYLE>
BODY {background-color:lightblue; color:red; margin-left:0.5in; margin-right:0.5in}
H1 {background-color:yellow; colored}
H2 {color:blue}
P {font-size:14pt; color:green;}
</STYLE>
```

This style is created and saved in a file named as Style Less.

Creating the Link

```
<HTML>
<Head>
<Title>Link</Title>
<link rel=stylesheet href= "Style1.css" type= "text/css">
</Head>
</HTML>
```

And the output will be same as embedded style sheet. It will set the style tags from the link file and change the HTML document.

13.6.4. STYLE SHEET PROPERTIES

(CSS1) cascading style sheet level 1 defines more than 50 different properties and values. This section covers standard CSS1 Properties, which should work in all browsers. Although they should work, some properties may not work in your browser due to minor rendering differences.

1. Font Properties

The following is not a full list, but only the most common properties are :

- (a) font-family — Denotes font
- (b) font-size — Denotes the size of the text
- (c) font-style — Denotes the style of the text for example normal, bold, italic etc.
- (d) font-weight — Denotes the weight or darkness of the font. This value ranges from 100 to 900 increments of 100.

2. Text Properties

- (a) **Letter-spacing.** Denotes the space between letters the default value of A, variety of measurements like inches (in), centimeters (cm), millimeter (mm), points (pt) etc., can be used.
- (b) **Word-spacing.** Denotes the space between words. Measurements can be used similar to letter spacing.

- (c) **Vertical-Align.** Denotes the vertical positioning of the text and images, with respect to the baseline. The possible values include baseline, sub, super, top, text-top middle percentage values etc.
- (d) **Text-align.** Specifies the alignment of the text. The possible values are center, justify etc.
- (e) **Text-indent.** Denotes margins. This property sets the indentation for text in the first line of block level of element.
- (f) **Text-transform.** Denotes the transformation of text. The possible values are capitalize, uppercase, lowercase etc.
- (g) **Text-decorate.** Denotes the text's decoration. The standard values for this property include blink, line-through overline underline etc.

3. Colour and Background Properties

CSS1 supports a variety of properties that can be used to control the colours and the backgrounds in a document. With style sheets you can create arbitrary regions with different background colours and images.

- (a) **Colour :** Denotes colour property, used to set text colour.
- (b) **Background-colour :** This property sets an element's background colour.
- (c) **Background-image :** Associates a background image with an element.
- (d) **Background-position :** Specifies how a background image is positioned.

4. Box Properties

Block style elements such as the <P> element can be considered as rectangular boxes on the screen. These box properties include :

- (a) **Margin-properties :** The margin values should be in length like 1.5 pt. etc. The individual margins for a block element can be set using margin-top, margin-right, margin-bottom, or margin-left.
- (d) **Border properties :**
 - **Border-style :** Used to set the appearance of the borders. The values can include solid, double, groove, ridge etc.
 - **Border-width :** The width of the border is specified here. The individual border width can be set using border-top width etc. border-right-width, border-bottom-width border-left-width etc.
 - **Border-colour:** Border may be assigned a colour.

5. Padding Properties

The space between an element's border and its content can be specified four padding regions can be set using the padding-top, padding-right, padding-bottom, and padding-left properties.

6. Microsoft Specific Style Sheet Properties

Microsoft has added a variety of multimedia filters and transitions that can be accessed via a style sheet. The basic form of a filters rule is shown below :

Filter: Filter name (Filter value 1, filter value 2)

Each of the possible values for filter name is shown in the table below :

Filter	Description
Alpha	Sets a uniform transparency level
Blur	Creates the impression of moving at high speed
Chroma	Makes a specific colour transparent
Dropshadow	Creates a solid silhouette of the object
FlipH	Creates horizontal mirror image
FlipV	Creates vertical mirror image
Glow	Add radiance around the outside edges of the object
Grayscale	Drops colour information from the image
Invert	Reverses the hue, saturation and brightness values
Light	Projects a light source into an object
Mask	Creates a transparent mask from an object
Shadow	Creates an offset solid silhouette
Wave	Creates a sine wave distortion along the X-axis
Xray	Shows just the edges of the object

13.6.5. ADVANTAGES OF CSS.

1. **CSS saves time.** When most of us first learn HTML, we get taught to set the font face, size, colour, style etc every time it occurs on a page. This means we find ourselves typing (or copying & pasting) the same thing over and over again. With CSS, you only have to specify these details once for any element. CSS will automatically apply the specified styles whenever that element occurs.
2. **Pages load faster.** Less code means faster download times.
3. **Easy maintenance.** To change the style of an element, you only have to make an edit in one place.
4. **Superior styles to HTML.** CSS has a much wider array of attributes than HTML.
5. **Search Engine Optimization.** When you incorporate CSS into your website design, you will soon discover that search engines will have the aptitude to locate them much

easier since CSS features a coding technique that is clear and simple to read. Search engines will no longer have to struggle to comprehend your website content. The advantage of improved accessibility of content will allow a broader number of users to locate you on the Internet. Less code and more content equals search engine success.

6. **Browser Compatibility.** Browser compatibility is very important and CSS addresses this issue nicely. When you decide to use CSS, you will find that, it will improve the characteristic of your website while securing your visitors with the capacity to view your website as precisely as you have designed it to be.

7. **Appearance.** CSS makes it easy to improve appearance of a website by allowing you to create a much more stylish website since CSS offers a wide array of expressive style capabilities. With CSS, you will actually obtain more control over your website's visual aspect.

8. **Bandwidth Savings.** Once CSS takes apart your website's content from its design language, you will trim down your file transfer magnitude significantly. These bandwidth savings are considerable figures of insignificant tags that are discredited from a multitude of pages. This will leave less, but more significant captions, listings and paragraphs.

13.6.6. DISADVANTAGE OF CSS

Browser compatibility: Browsers have varying levels of compliance with Style Sheets. This means that some Style Sheet features are supported and some are not. Some browser manufacturers decide to come up with their own proprietary tags. Fortunately browser compatibility is becoming less of an issue as the latest browser versions are much more standards-compliant than their earlier counterparts.

13.7 JavaScript Style Sheets (JSSS)

JavaScript Style Sheets (JSSS) was a style sheet language technology proposed by Netscape Communications Corporation in 1996 to provide facilities for defining the presentation of webpages. It was an alternative to the Cascading Style Sheets (CSS) technology. Although Netscape submitted it to the World Wide Web Consortium (W3C), the technology was never accepted as a formal standard and it never gained much acceptance in the market.

13.7.1. JAVASCRIPT STYLE SHEET PROPERTIES

A subset of the CSS style properties are supported in JSSS. The JSSS style properties, their CSS equivalents are shown in Table 1 below.

Table 1: JavaScript Style Sheet properties and their Cascading Style Sheet equivalents

JSSS Property	CSS Property
align	float
backgroundImage	background-image
backgroundColor	background-color
borderBottom Width	border-bottom-width
borderLeftWidth	border-left-width
borderRightWidth	border-right-width
borderStyle	border-style
borderTop Width	border-top-width
clear	clear
display	display
fontSize	font-size
fontStyle	font-style
height	height
lineHeight	line-height
listStyleType	list-style-type
marginBottom	margin-bottom
margin Left	margin-left
marginRight	margin-right
marginTop	margin-top
paddingBottom	padding-bottom
paddingLeft	padding-left
paddingRight	padding-right
paddingTop	padding-top
textDecoration	text-decoration
textTransform	text-transform
text Align	text-align
textIndent	text-indent
verticalAlign	vertical-align
whiteSpace	white-space
width	width

JSSS also defines three methods that allow you to define margins, padding, and border widths within a single style property. The three methods, margins(), paddings(), and border Widths(), accept four parameters, corresponding to the top, right, bottom, and left margin.

padding or border width, respectively. Unlike their CSS counterparts the JSSS methods require that you always specify all four parameters. There is no shorthand way in JSSS to set multiple margins, paddings, or border widths with a single value.

13.7.2. DIFFERENT WAYS TO ATTACH JSSS TO TAG

There are three different methods to attach JSSS's to tag. These are Inline Styles: The style Attribute, Document-Level Style Sheets and External Style Sheets.

1. Inline Styles: The style Attribute

The inline style is the simplest way to attach a style to a tag—just include a style attribute with the tag along with a list of properties and their values. The browser uses those style properties to render the contents of just this instance of the tag.

For instance, the following style tells the browser to display the level-one header text, "I am a student", not only in the <h1> tag style characteristic of the browser, but also in the color blue and italicized.

```
<h1 style="color: blue; font-style: italic">I am a Student </h1>
```

This type of style definition is called "inline" because it occurs with the tag as it appears in the document. The scope of the style covers the contents of that tag only. Since inline styles are sprinkled throughout the document, they can be difficult to maintain. Use the style attribute sparingly and only in those rare circumstances when you cannot achieve the same effects otherwise.

2. Document-Level Style Sheets

The real power of style sheets dawns when you place a list of presentation rules within the head of an HTML document. Enclosed within their own <style> and </style> end tags, so-called "document-level" style sheets affect all the same tags within that document, except for tags that contain an overriding inline style attribute.

<Style>

Function:

Defines a document-level style sheet

Attributes:

TYPE

End tag:

</style>; rarely omitted

Contains:

styles

Used in:

head_content

The <style> tag must appear within the <head> of a document. Everything between the <style> and </style> tags is considered part of the style rules to be applied to the document.

To be perfectly correct, the content of the <style> tag is not HTML and is not bound by the normal rules for HTML content. The <style> tag, in effect, lets you insert foreign content into the HTML document that the browser uses to format the tags. The <style> tag has just one attribute, type. It defines the types of styles you are including within the tag. Cascading style sheets always carry the type text/css; JavaScript style sheets use the type text/javascript. You may omit the type attribute and hope the browser will figure out the kind of styles you are using. Include the type attribute so that there is no opportunity for confusion.

For example, a style-conscious browser will display the contents of all <h1> tags as blue, italic text in a document that has the following document-level style sheet, definition in its head:

```
<head>
<title>All True Blue</title>
<style type="text/css">
<!--
/* make a.11 H1 headers blue */
H1 {color: blue; font-style: italic}
-->
</style>
</head>
<body>
<h1>I am a student </h1>
<h1>I am a ^Teacher </h1>
```

3. External Style Sheets :

You may also place style definitions, like the document-level style sheet example for <h1> tags, in a text file with the MIME type of text/css and import this "external" style sheet into the HTML documents. Because an external style sheet is a file separate from the HTML document and is loaded by the browser over the network, you can store it anywhere, reuse it often, and even use others style sheets. The external style sheets give you the power to influence the display styles not only of all related tags in a single document, but for an entire collection of documents.

For example, suppose we create a file named gen_styles.css containing the style rule:

```
H1 {color: blue; font-style: italic}
```

For each and every one of the HTML documents in the collections, we can tell the browser to read the contents of the gen_styles.css file, which in turn will color all the <h1> Mag contents blue and render the text in italic. Of course, that will be true only if the user's machine is capable of these style tricks, they are using a style-conscious browser, and the style isn't overridden by a document-level or inline style definition.

The external style sheets can be loaded into the HTML document in two different ways: linked and imported.

(a) Linked external style sheets

One way to load an external style sheet is to use the <link> tag:

```
<head>
<title>Style linked</title>
<link rel=stylesheet type="text/css"
      href="http://www.kumquats.com/styles/gen_styles.ess"
      title="The blues">
</head>
<body>
<h1>I am a student</h1>
<h1> I am a Teacher </h1>
```

Recall that the <link> tag creates a relationship between the current document and some other document on the Web. In the example, we tell the browser that the document named in the href attribute is a stylesheet, and that its contents conform to the CSS standard, as indicated by the type attribute. We also provide a title for the style sheet, making it available for later reference by the browser.

The <link> tag must appear in the <head> of a document. The URL of the style sheet may be absolute or relative to the document's base URL. The type may also be text/javascript, indicating (for Netscape only) that the style rules are written in JavaScript instead of the CSS syntax.

(b) Imported external style sheets

The second technique for loading an external style sheet imports the files with a special command within the <style> tag:

```
<head>
<title>Imported style sheet</title>
<style>
<!--
      @import url(http://www.kumquats.com/sty les/gen_styles. ess);
      @import url(http://www.kumquats.com/styles/spec_styles.css);
      BODY: (background: url(backgrounds/marble.gif))
</style>
</head>
```

The @import command expects a single URL parameter that names the network path to the external style sheet. The url keyword, parentheses, and trailing semicolon are all required elements of the @import command. The URL may be absolute or relative to the document's base URL. The @import command must appear before any conventional style rules, either in the <style> tag or in an external style sheet. Otherwise, the browser ignores

the preceding style definitions. This ordering also means that subsequent style rules can override rules in the imported sheet, and indeed they do.

The import command can appear in a document-level style definition or even in another external style sheet, letting you create nested style sheets.

(c) Linked Versus Imported Style Sheets

At first glance, it may appear that linked and imported style sheets are equivalent, using different syntax for the same functionality. This is true if you use just one <link> tag in the document. However, special rules come into play if you include two or more <link> tags within a single document.

With one <link> tag, the browser loads the styles in the referenced style sheet and formats the document accordingly, with any document-level and inline styles overriding the external definitions. With two or more <link> tags, the browser presents the user with a list of all the <link>-ed style sheets. The user can then select one of the sheets, which is used to format the document. The other <link>-ed style sheets are ignored.

On the other hand, the style-conscious browser merges, as opposed to separating, multiple @imported style sheets to form a single set of style rules for the document. The last imported style sheet takes precedence if there are duplicate definitions among the style sheets. Imported styles override linked external styles, just as document-level and inline styles override external style definitions. To bring this all together, consider the example:

```
<html>
<head>
<link rel=stylesheet href=sheet1.ess type=text/css>
<link rel=stylesheet href=sheet2.css type=text/css>
<style>
<!--
      @import url(sheet3.css);
      @import uri(sheet4.css);
-->
</style>
</head>
```

Using the CSS model, the browser will prompt the user to choose between sheet Less and sheet2.css. It will then load the selected sheet, followed by sheet3.css and sheet4.css. Duplicate styles defined in sheet3.css or sheet4.css and in any inline styles will override styles defined in the selected sheet.

13.7.3. WORKING WITH JSSS

Lets create style sheets using JSSS objects. The three main objects that JSSS handles are Tags, Classes and ids.

Example 3. This program shows the use of Cascading Style Sheet.

```
<Html>
<Head>
<Title>Text Formatting</Title>
</Head>
<Style>
Body {Background: pink; color: blue; margin-left: 0.5in;
margin-right: 0.5in; H1 {font-size:26; font-family: Monotype Corsiva; color: red;}
H2 {font-size:24; color: yellow; font-weight:bold; background: red}
P {font-size: 18pt; font-style: italic;}
</Style>
<Body>
<Center><H1>Internet</H1></Center>
<P>Internet connection is now available at Rs.1000 for 100 hrs. For 50 hrs we are
charging Rs.500 and for 25 hrs Rs.250</P>
<Center><H2>Diwali Offer</H2></Center>
<P>50 hrs free connection with 100 hrs connection. 20 hrs free with 50 hrs connection
and 10 hrs free with 25 hrs connection.
</P>
</Body>
</Html>
```

Now the same program is written again using JSSS i.e. Java Script Style Sheet. Note the difference between the scripting tags.

```
<Html>
<Head>
<Title>Text Formatting using JSSS</Title>
</Head>
<Style type="text/javascript">
with(tags.Body) {
background-color="pink";
color="blue";
margin-left="50";
margin-right="50";
} with(tags.H1)
font-size="26pt";
```

```
font-family="Monotype Corsiva";
color="red";
}
with(tags.H2) {
font-size="24pt";
font-weight="bold";
background-color="red";
color="yellow";
}
with(tags.P) {
font-size="18pt";
font-style="italic";
}
</Style>
```

```
<Body>
<Center><H1>Internet</H1></Center>
<P>Internet connection is now available at Rs.1000 for 100 hrs.
For 50 hrs we are charging Rs.500 and for 25 hrs Rs.250.</P>
<Center><H2>Diwali Offer</H2></Center>
<P>50 hrs free connection with 100 hrs connection.
20 hrs free with 50 hrs connection and 10 hrs free with 25 hrs connection.
</P>
```

```
</Body>
</Html>
```

For example: Viewing an item in a paragraph.

```
<Center><H1>Music Program</H1></Center> <div id="layer1";
style="position:static; font-size:20; color:blue">
<P>We are going to organize a Music Program at Netaji Indoor Stadium on 25<Sup>th
</Sup>December.
So join us and enjoy the Christmas Night.</P>
<div id="layer2"
style="position:static; font-size: 18; background-color: pink; color: blue">
<Center><P>The program is organized by Young Bengal Students Group.</P></div>
Center>
```

```

</div>
<div id="layer3"
style="position:static; Left:50; Top:200; Width:300; Height:100; Font-style:italic;
Font-weight:bold; Font-size:16; background-color:yellow; color:red">
<UL>
<LI>Holy Band Group
<LI>Children's Band Group
<LI>Teen's Band Group
</UL>
<div id="layer4"
style="position:absolute; Left:400; Top:250; Height:100; Font-weight:bold; font-
size:14; background-color:white">
(PinS <P>The Names of the Participants</P>
<OL>
<LI>Rahul
<LI>Minu
<LI>Tina
<LI>Mohan
</OL>
<div id="layer5"
style="position:absolute; Left:100; Height:100; Width:200; Top:300; Font-weight:bold;
Font-size:18; background-color:orange; color:blue">
Tickets available at Counter of Rs.200(Front Seat), Rs.150(Chair), Rs.100 (Gallery).
<div id="layer6" *
style="position:absolute; Left:400; Height:70; Width:150; Top:400; Font-size:20;
background-color:blue; color:white">
So. Hurry Up!
</Body>
</Html>
</enter>

```

13.7.4. LAYERS OF NETSCAPE

The layers are basically two dimensional boxes of content which can be positioned absolutely on a page. The layers can be overlapped and using JavaScript can be moved around, hidden or revealed.

Layers were introduced with the Netscape-specific <layer> tag, but it was never adopted by Microsoft and is not an officially (WWW) recognised part of HTML. However, precisely the same functionality is available to browsers supporting CSS using CSS-positioning, particularly as applied to the <div> element. This works in both Netscape and Internet Explorer. The Netscape-specific <layer> tag is rarely used.

USING LAYERS

In order to make a layer, you use the <LAYER> or <ILAYER> tag. Each has its corresponding (</LAYER> or </ILAYER>) closure. Between the tag and its closure, you put HTML code that goes to make up the layer. The good thing is that you can then freely move the layer around - over or under the other layers that make up the document. An easy way to say which layer goes over what is simply to use a Z-INDEX attribute for each layer. The difference between a layer and an ilayer is that the layer is 'fixed' (and separate), the ilayer is part of the natural flow of a document.

13.7.4.1 Properties of Layers:

- 1. Layer transparency.** Layers are by default transparent, if you want a layer to be opaque, then simply specify a BGCOLOR in the LAYER tag. But remember that if you do this, all layers behind this one will be invisible! You can also make only part of a layer opaque - look at our example again, and move the red ball down and right - see how the central controls have an opaque background, because we specified a background colour of white for the table that contains them! You can also specify a background image for a layer - just say BACKGROUND="img.gif" or whatever in the layer tag. You can specify layer visibility using visibility="show" instruction inside the layer tag.
- 2. Positioning a layer:** You can move a layer around and also you can specify its starting position. You can do so by specifying the TOP and LEFT coordinates of the layer - these position the top left corner at the stated pixel coordinates.
- 3. Limiting the layer:** There are several ways of limiting the layer's extent:
 1. You can define its width (in pixels or as a percent of screen width) using (for example) width="20%";
 2. You can define its clipping rectangle. This takes four arguments thus: clip="20,50,150,100"

The above line will take the layer as it would normally be displayed, and then clip 20 pixels off the left margin, 50 off the top, 150 off the right and 100 off the bottom margins, there's another infernally helpful format:

clip="150, 100"

- if you include ; ust two numbers, the-: assumption is that they specify the right and bottom clipping margins, with the left and top margins set to zero. The clip object has six properties: clip.bottom, clip.top, clip.left, clip.right as well as clip.height and clip.width.

13.7.4.2 Attributes used in Layer :

The following are the attributes that you can use in a layer:

Layer attributes (and object properties)	
Attribute (an example)	Attribute (an example)
bgColor = "#FFFFFF"	Set background colour to opaque, and white. Similar is background which is used to set a background image.
clip="20,50,150,100"	Set clipping rectangle as described above. (The corresponding object has six properties - top, left, bottom, right, height, width)
left="25"top="2%"	Set offset of layer in pixels or percentage - top is from top margin, left from left margin of parent layer. Synonyms for the properties Layer.left and Layer.top are x and y!
visibility="show"	layer is visible, the converse is "hide"; another option is visibility- show "inherit".
width="20%"	width of frame (as percent of parent, or in pixels). Width is not directly a property of the layer object, but rather its clip property there is no Layer.width property.
z-index="1"	A positive integer which says which frame is in front (frames with bigger z-indices are in front of those with smaller). Similar are above and below which identify the frame objects above and below the current one. Note that the corresponding property of the layer object is zIndex, and not I z-index.
src="somewhere.htm"	Specify the source from which html code is loaded into the src-somewhere.htm frame.
id="fred"	The name property of the layer is specified using the attribute. It would appear that one can also say name="fred" = the documentation is a little unclear.
	In addition there are JavaScript properties of layers that are not represented in the attributes of the <LAYER> tag. These are:
parentLayer	The parent layer that contains this layer, or if there is none, parentLayer then the enclosing window object.
siblingAbove	If this layer has a parent, and the parent has other child layers, then these children are siblings of the current layer. siblingAbove The siblingAbove is the sibling immediately in front of the current layer (ie with z-order 1 greater than that of the current layer).

siblingBelow	like siblingAbove, but the sibling layer with lower z-order.
document	document the document contained in the frame.
window	The window (or frame) object that contains the layer, window regardless of nesting of layers.
pageX, pageY	the horizontal and vertical pixel positions of the layer.

13.7.4.3 Layer Methods :

There are also several methods you can use to alter the properties of a layer. They are:

Layer Methods	
Method	What it does
moveAbove (layer)	move the layer above the specified layer
moveBelow	as for move Above
	move the layer to the specified positionh - (0,0) is the top left hand corner of the screen; moveBy() is the corresponding function for a relative.
moveTo(x,y)	movement moveToAbsolute() moves the layer to the specified page coordinates, unlike all the other move methods, which are relative to the containing layer.
offset(x,y)	relative movement (in pixels).
resize(w,h)	resize the layer to the stated width and height (in pixel). They have resizeBy() (which effectively adds/subtracts width/height to/from clip. width and clip. height;
captureEvents(),releaseEvents(),handleEvent(),routeEvent()	
load()	load the given URL into the layer. The wrapping width of the layer content is apparently also changed.

Example 4. This program shows the functionality of layers

```
<Html>
<Head>
<Title>Layer</Title>
</Head>
<Body>
```


13.8 The ID Attribute

The *id* attribute can be used to uniquely identify any element within a page (or style sheet). There are two primary reasons that you might want to use an *id* attribute on an element:

- If an element carries an *id* attribute as a unique identifier it is possible to identify just that element and its content.
- If you have two elements of the same name within a Web page (or style sheet), you can use the *id* attribute to distinguish between elements that have the same name.

We v-viii! discuss style sheet in separate tutorial. For now, the *id* attribute could be used to distinguish between two paragraph elements, like so:

```
<p id="html">This para explains what is HTML</p>
```

```
<p id="css">para explains what is Cascading Style Sheet</p>
```

Note that there are some special rules for the value of the *id* attribute, it must:

- Begin with a letter (A-Z or a-z) and can then be followed by any number of letters, digits (0-9), hyphens, underscores, colons, and periods.
- Remain unique within that document; no two attributes may have the same value within that HTML document.

13.9 DHTML Events:

DHTML events are an incident which occurs in a particular place during a particular interval of time. Clicking a button, moving the mouse pointer over part of the Web page, selecting some text on the page, these actions all raise events, and a DHTML programmer can write code to run in response to the event. Examples of events:

- A mouse click
- A web page or an image loading
- Mousing over a hot spot on the web page
- Selecting an input field in an HTML form
- Submitting an HTML form
- A keystroke

See section 8.3.1 from chapter 8 for more details.

Solved Questions

1. What are Style Sheets?

Ans. Style Sheets are templates, very similar to templates in desktop publishing applications, containing a collection of rules declared to various selectors (elements).

2. What is CSS?

Ans. CSS stands for Cascading Style Sheets and is a simple styling language which allows attaching style to HTML elements. Every element type as well as every occurrence of a specific element within that type can be declared an unique style, e.g. margins, positioning, color or size.

2. CSS is a web standard that describes style for XML/HTML documents.

3. CSS is a language that adds style (colors, images, borders, margins...) to your site. CSS is not used to put any content on your site, it's just there to take the content you have and make it pretty. First thing you do is link a CSS-file to your HTML document. This can be done as

```
<link rel="stylesheet" href="style.css" type="text/css">
```

The line should be placed in between your `<head>` and `</head>` tags. If you have several pages you could add the exact same line to all of them and they will all use the same style sheet. Let's look inside the file "style. css" we just linked to.

```
h1 {
  font-size: 40px;
  height: 200px;
}
warning {
  color: Red;
  font-weight: bold;
}
#footer {
  background-color: Gray;
}
```

4. Cascading Style Sheets (CSS) is a simple mechanism for adding style (e.g. fonts, colors, and spacing) to Web documents. CSS helps you spice up your web-page and make it look neat in wide variety of aspects.

3. What are Cascading Style Sheets?

Ans. A Cascading Style Sheet (CSS) is a list of statements (also known as rules) that can assign various rendering properties to HTML elements. Style rules can be specified for a single element occurrence, multiple elements, an entire document, or even multiple documents at once, it is possible to specify many different rules for an element in different locations using different methods. All these rules are collected and merged when the document is rendered to form a single style rule for each element.

4. What is CSS rule 'ruleset'?

Ans. There are two types of CSS rules: ruleset and at-rule. Ruleset identifies selector or selectors and declares style which is to be attached to that selector or selectors. For example P (text-indent: 10pt) is a CSS rule. CSS rulesets consist of two parts: selector, e.g. P and declaration, eg. {text-indent: 10pt}

P (text-indent: 10pt) - CSS rule (ruleset)

(text-indent: 10pt) - CSS declaration

text-indent - CSS property

10pt-CSS value

5. What is external Style Sheet? How to link?

Ans. External Style Sheet is a template/document/file containing style information which can be linked with any number of HTML documents. This is a very convenient way of formatting the entire site as well as restyling it by editing just one file. The file is linked with HTML documents via the LINK element inside the HEAD element. Files containing style information must have extension .CSS, e.g. style.css. <HEAD> <LINK REL=STYLESHEET HREF="style.css" TYPE="text/css"> </HEAD>

6. Is CSS case sensitive?

Ans. Cascading Style Sheets (CSS) is not case sensitive. However, font families, URLs to images, and other direct references with the style sheet may be.

7. Why do style sheets exist?

Ans. SGML (of which HTML is a derivative) was meant to be a device-independent method for conveying a document's structural and semantic content (its meaning.) It was never meant to convey physical formatting information. HTML has crossed this line and now contains many elements and attributes which specify visual style and formatting information. One of the main reasons for style sheets is to stop the creation of new HTML physical formatting constructs and once again separate style information from document content.

8. Which browsers support CSS?

Ans. The following browsers support CSS.

- * Internet Explorer 3.0 and above
- * Navigator 4.0 and above

- * Opera 3.6 and above etc.

9. What is alternate Style Sheet? How to link?

Ans. Alternate Style Sheet is a sheet defining an alternate style to be used in place of style(s) declared as persistent and/or preferred.

Persistent style is a default style that applies when style sheets are enabled but can be disabled in favor of an alternate style, e.g.:

```
<LINK REL=Stylesheet HREF="style.css" TYPE="text/css">
```

Preferred style is a default style that applies automatically and is declared by setting the TITLE attribute to the LINK element. There can only be one preferred style, e.g.:

```
<LINK REL=Stylesheet HREF="style2.css" TYPE="text/css" TITLE="appropriate style description">
```

Alternate style gives an user the choice of selecting an alternative style - a very convenient way of specifying a media dependent style. Note: Each group of alternate styles must have unique TITLE, e.g. >

```
<LINK REL="Alternate Stylesheet" HREF="style3.css" TYPE="text/css" TITLE="appropriate style description" MEDIA=screen>
```

```
<LINK REL="Alternate Stylesheet" HREF="style4.css" TYPE="text/css" TITLE="appropriate style description" MEDIA=print>
```

10. What is imported Style Sheet? How to link?

Ans. Imported Style Sheet is a sheet that can be imported to (combined with) another sheet. This allows creating one main sheet containing declarations that apply to the whole site and partial sheets containing declarations that apply to specific elements (or documents) that may require additional styling. By importing partial sheets to the main sheet a number of sources can be combined into one.

To import a style sheet or style sheets include the @import notation or notations in the STYLE element. The @import notations must come before any other declaration. If more than one sheet is imported they will cascade in order they are imported - the last imported sheet will override the next last; the next last will override the second last, and so on. If the imported style is in conflict with the rules declared in the main sheet then it will be overridden.

```
<LINK REL=STYLESHEET HREF="main.css" TYPE="text/css">
```

```
<STYLE TYPE="text/css">
```

```
<!--
```

```
@import url(http://www.and.so.on.partial1.css);
```

```
@import url(http://www.and.so.on.partial2.css);
```

```
..... other statements
```

```
-->
```

```
</STYLE>
```

11. How to use CSS to separate content and design?

Ans. The idea here is that all sites contain two major parts, the content: all your articles, text and photos and the design: rounded corners, colors and effects. Usually those two are made in different parts of a webpage's lifetime. The design is determined at the beginning and then you start filling it with content and keep the design fixed. In CSS you just add the <link>-tag. In the HTML document you put content only; and that link of yours makes sure it looks right. You can also use the exact same link on many of your pages, giving all of them the same design. You want to add content, then write a plain HTML document.

12. Can CSS be used with other than HTML documents?

Ans. Yes. CSS can be used with any structured document format, e.g. XML, however, the method of linking CSS with other document types has not been decided yet.

13. What is CSS declaration?

Ans. CSS declaration is style attached to a specific selector. It consists of two parts; property which is equivalent of HTML attribute, e.g. text-indent: and value which is equivalent of HTML value, e.g. 10pt. Properties are always ended with a colon.

14. Can Style Sheets and HTML stylistic elements be used in the same document?

Ans. Yes. Style Sheets will be ignored in browsers without CSS-support and HTML stylistic elements used.

15. Can comments be included in the Style Sheet?

Ans. Yes. Comments can be written anywhere where whitespace is allowed and are treated as white space themselves. Anything written between /* and */ is treated as a comment. Comments cannot be nested.

/* This is a CSS-comment */

16. How block-elements with CSS1 can be centered?

Ans. By setting the properties margin-left and margin-right to auto and width to some explicit value:

BODY (width: 30em; background: cyan;)

P (width: 22em; margin-left: auto; margin-right: auto)

In this case, the left and right margins will each be four ems wide, since they equally split up the eight ems left over from (30em — 22em).

It was not necessary to set an explicit width for the BODY element. For example. TABLE (margin-left: auto; margin-right: auto; width: 400 px;) In most browsers, a table's width is by default determined by its content.

Exercises

1. What is DHTML? What are its advantages?
2. What is DOM? What is object Hierarchy?

3. What are the features of DHTML? Explain.
4. Explain the different ways of implementing CSS.
5. Explain different ways to attach CSS to a tag.
6. Explain various properties of CSS.
7. What are the advantages and disadvantages of CSS?
8. What is JSS? What are its various properties?
9. Explain different ways to attach JSS to a tag.
10. What are layers in Netscape? What are its properties?
11. What is id attribute? Explain.
12. What do you mean by DHTML events? Explain.